

WHAT'S NEW IN RAD STUDIO 13.2 FLORENCE



RAD Studio 13.2 at a glance

IDE & Navigation

Integrated Navigator plugin alongside general IDE workflow improvements.

Linux Compiler Overhaul

Reworked Delphi compiler for better performance and stability on Linux.

Expanded Delphi LLVM20 Toolchain

Extended Delphi support for both Windows on Arm and Linux platforms.

C++ Win64x & Debugging

Higher quality build target with major 64-bit debugger updates.

UI & Style Designer

New VCL components and continued enhancements to the FireMonkey Style Designer.

Broad Ecosystem Quality

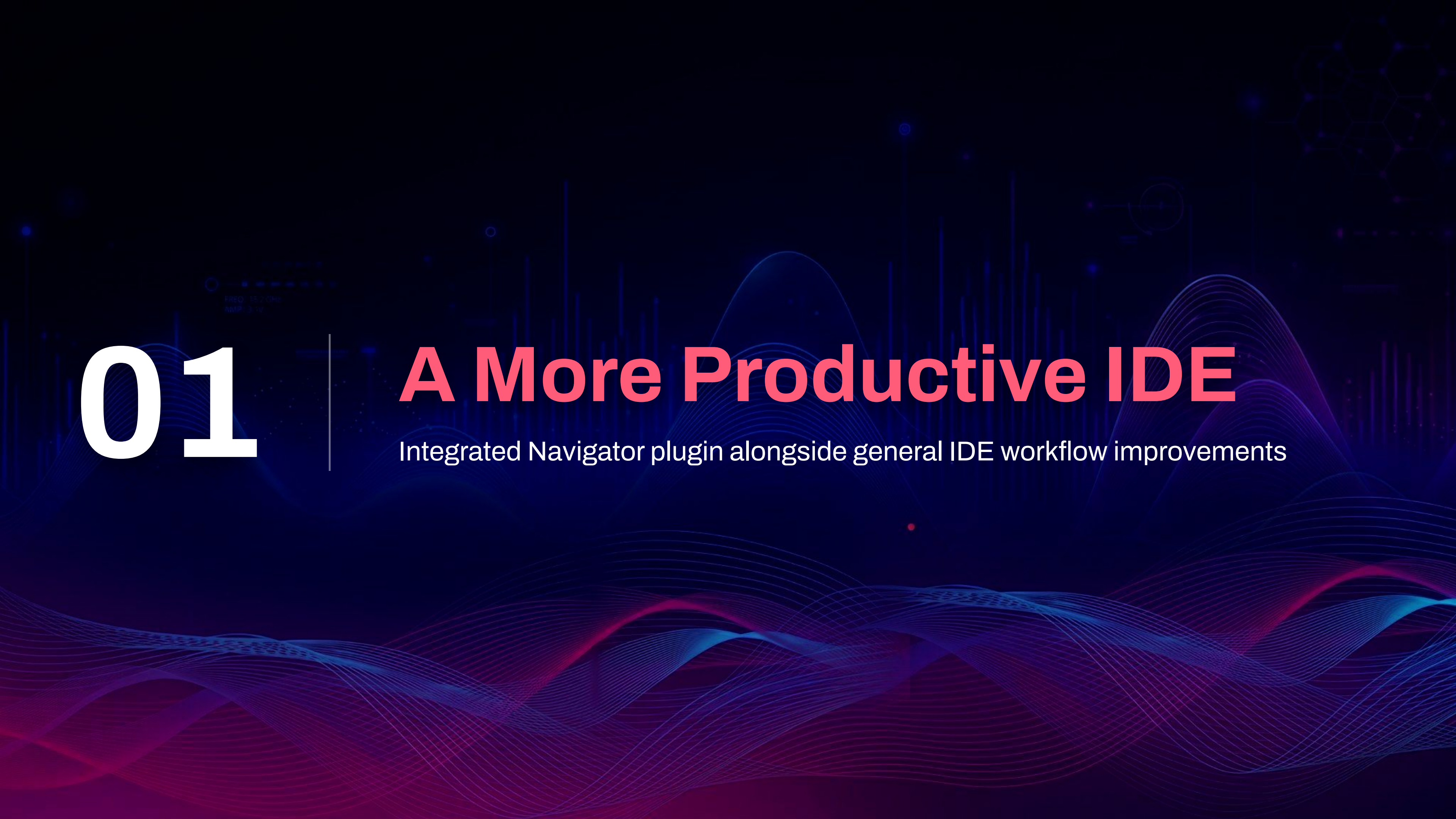
Focused extensions and quality work across mobile, database, web and backend.

MCP Server & AI

Official MCP server integration alongside improved SmartCore AI components.

Kai 1.1 Available

Fresh release introducing full ACP Registry support.



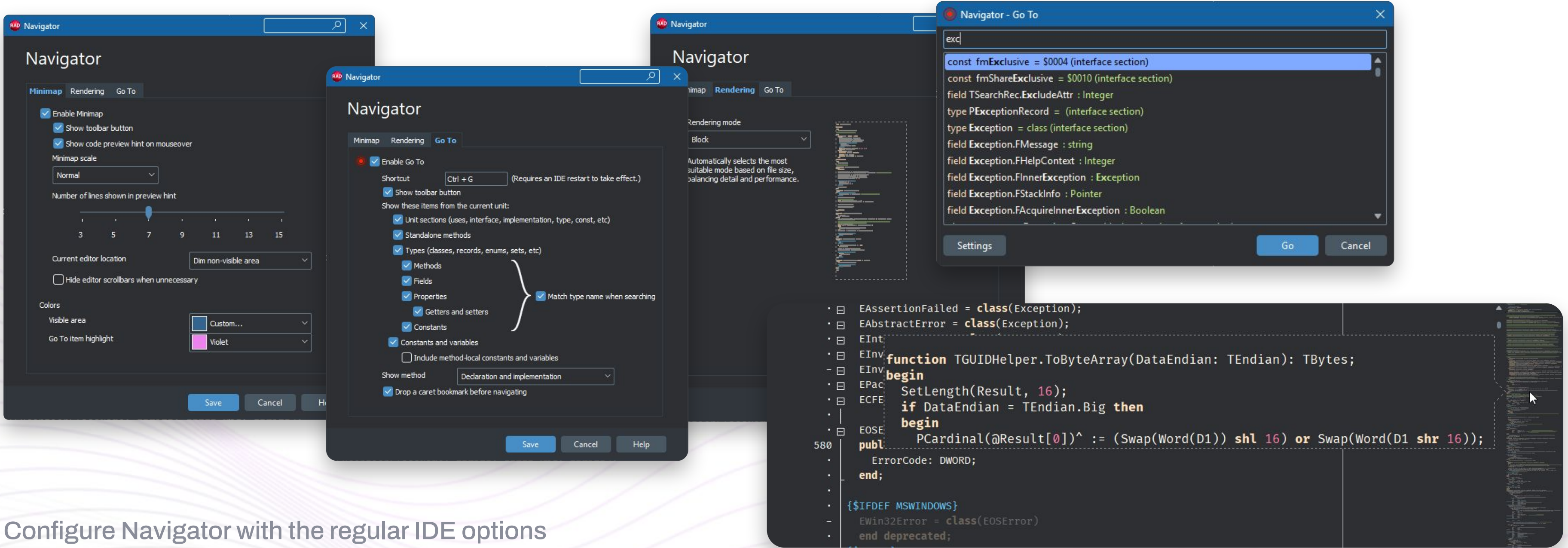
01

A More Productive IDE

Integrated Navigator plugin alongside general IDE workflow improvements

Navigate Large Projects Faster

Navigator is now fully integrated into RAD Studio

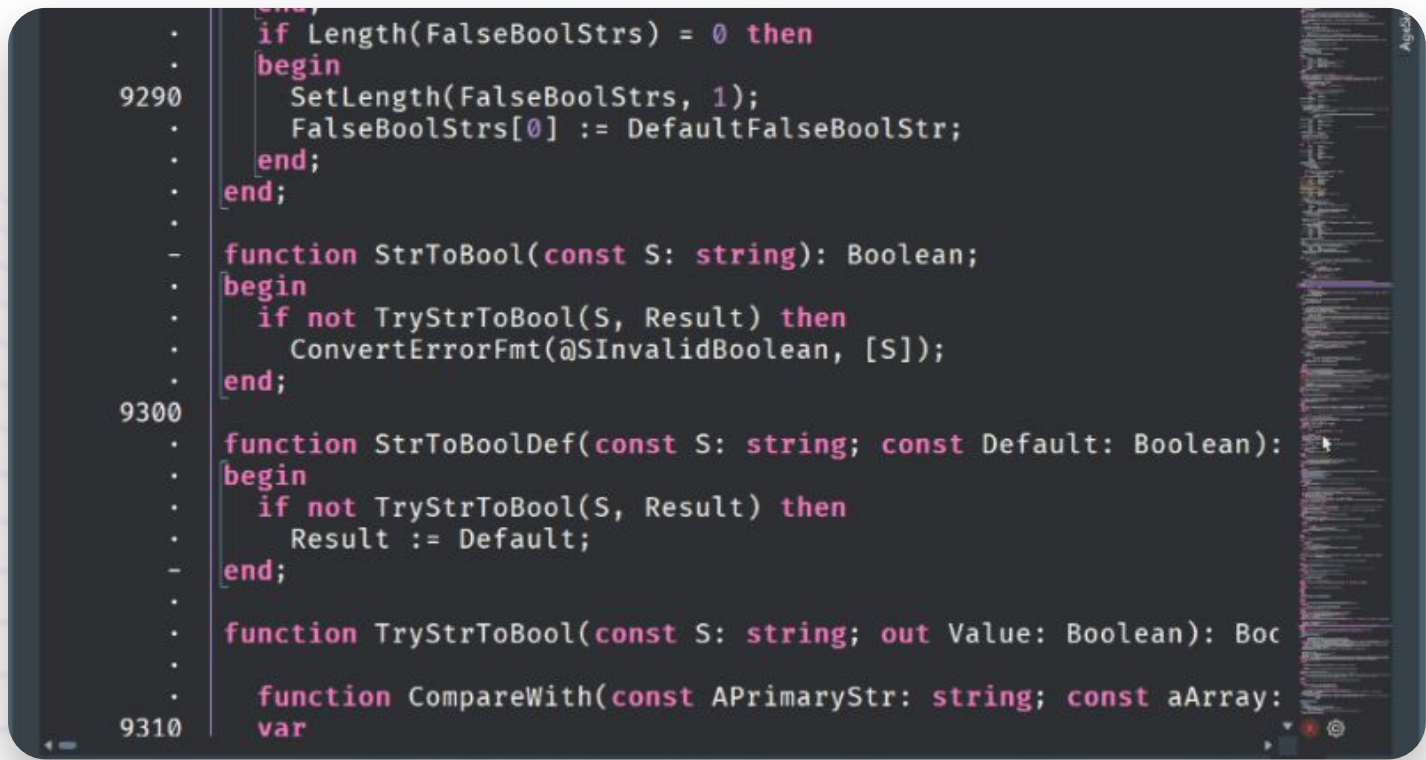


The image displays three overlapping screenshots of the RAD Studio Navigator interface. The leftmost screenshot shows the 'Navigator' window with the 'Minimap' tab selected, displaying settings for enabling the minimap, toolbar buttons, and preview hints. The middle screenshot shows the 'Navigator' window with the 'Go To' tab selected, displaying settings for enabling the 'Go To' feature, setting a shortcut (Ctrl + G), and configuring the items to be shown from the current unit. The rightmost screenshot shows the 'Navigator - Go To' dialog box, which lists search results for the text 'excl' in a project, including items like 'const fmExclusive', 'const fmShareExclusive', and 'type PExceptionRecord'. Below these screenshots, a portion of a Delphi code file is visible, showing a function 'TGUIDHelper.ToByteArray' and a class 'EWin32Error'.

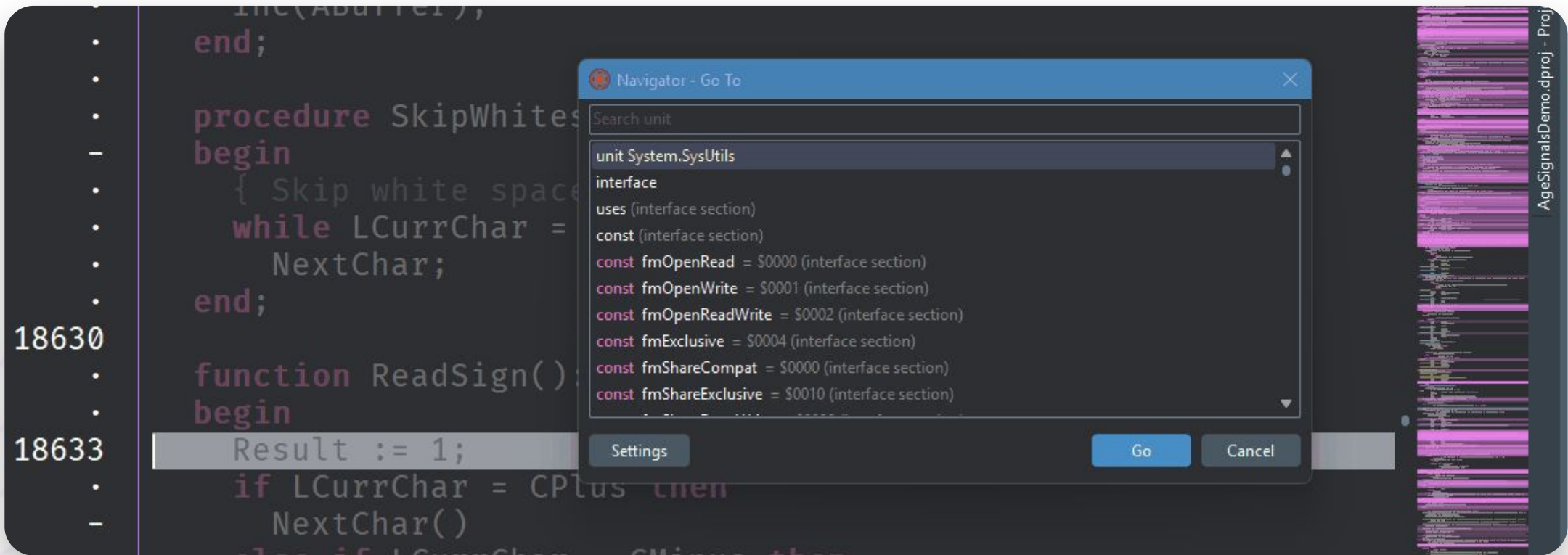
Configure Navigator with the regular IDE options

Navigate Large Projects Faster

- Search units, types, methods, and other symbols as you type
- Move directly to the relevant code without browsing individual files



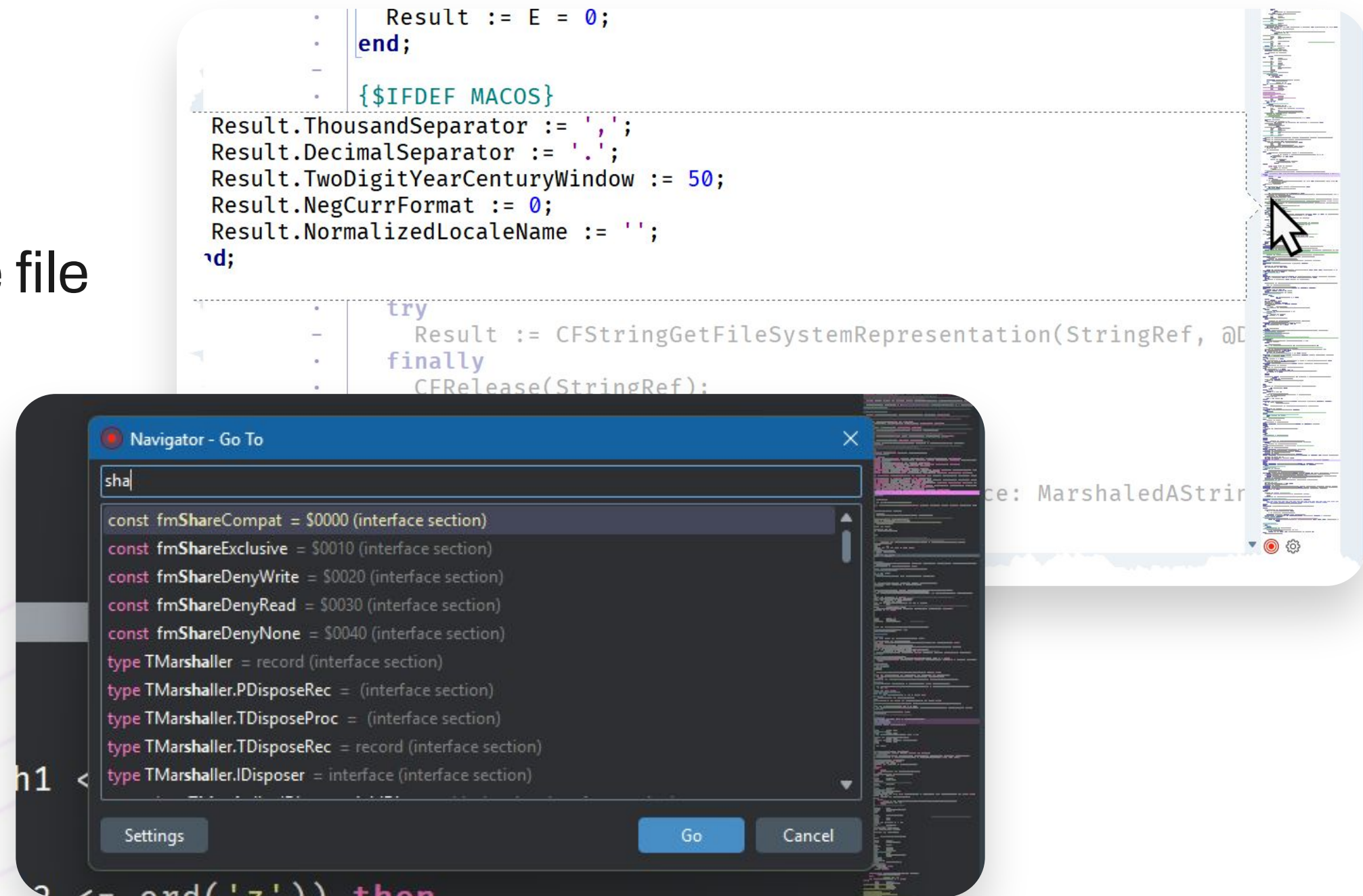
Minimap



Go to

The Navigator Minimap Adds Context

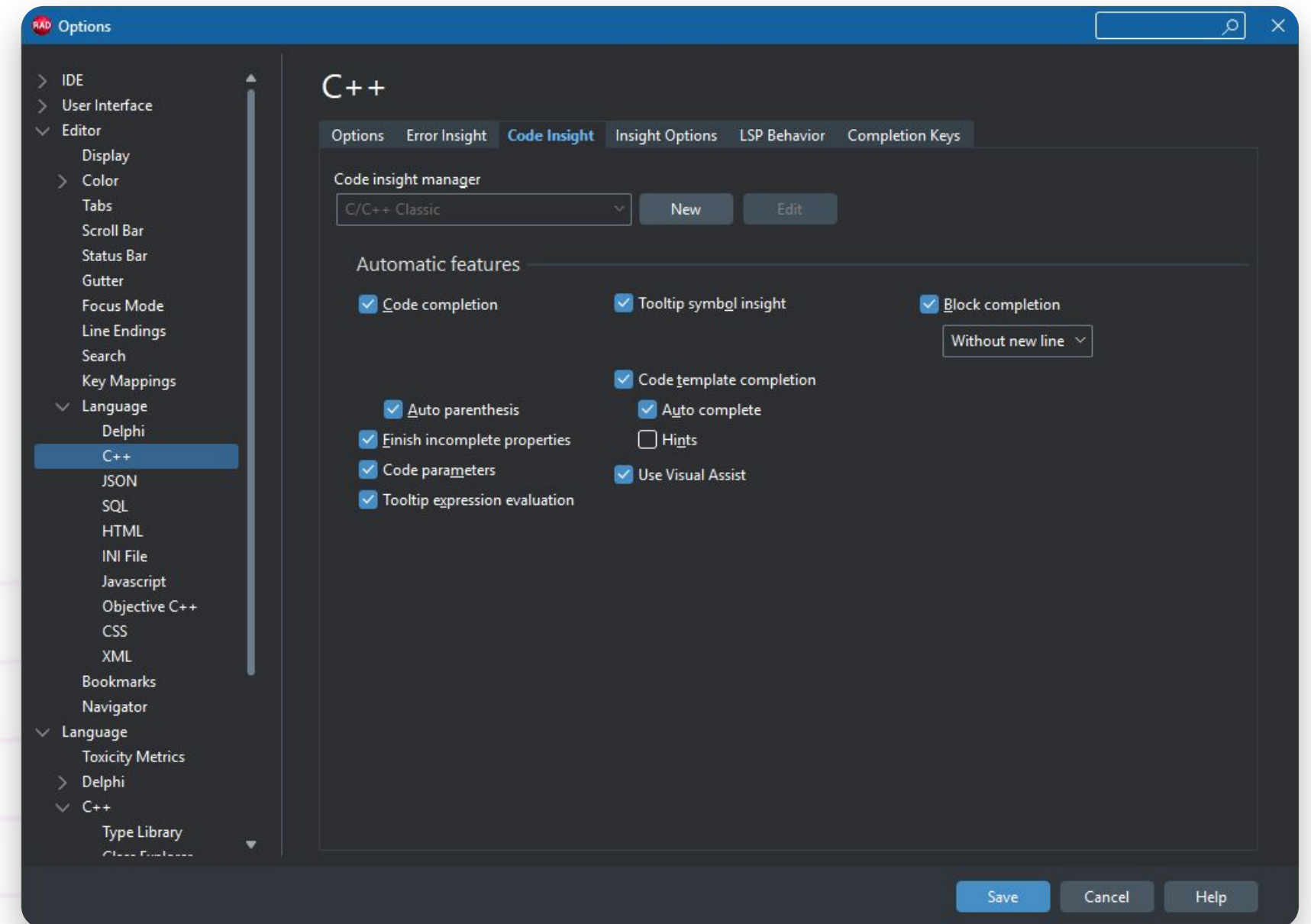
- See the structure of the complete source file
- Keep the active editor line visible
- Show breakpoints and Error Insight annotations



Use Goto and the minimap together for symbolic and visual navigation

Code Insight Settings Are Easier to Navigate

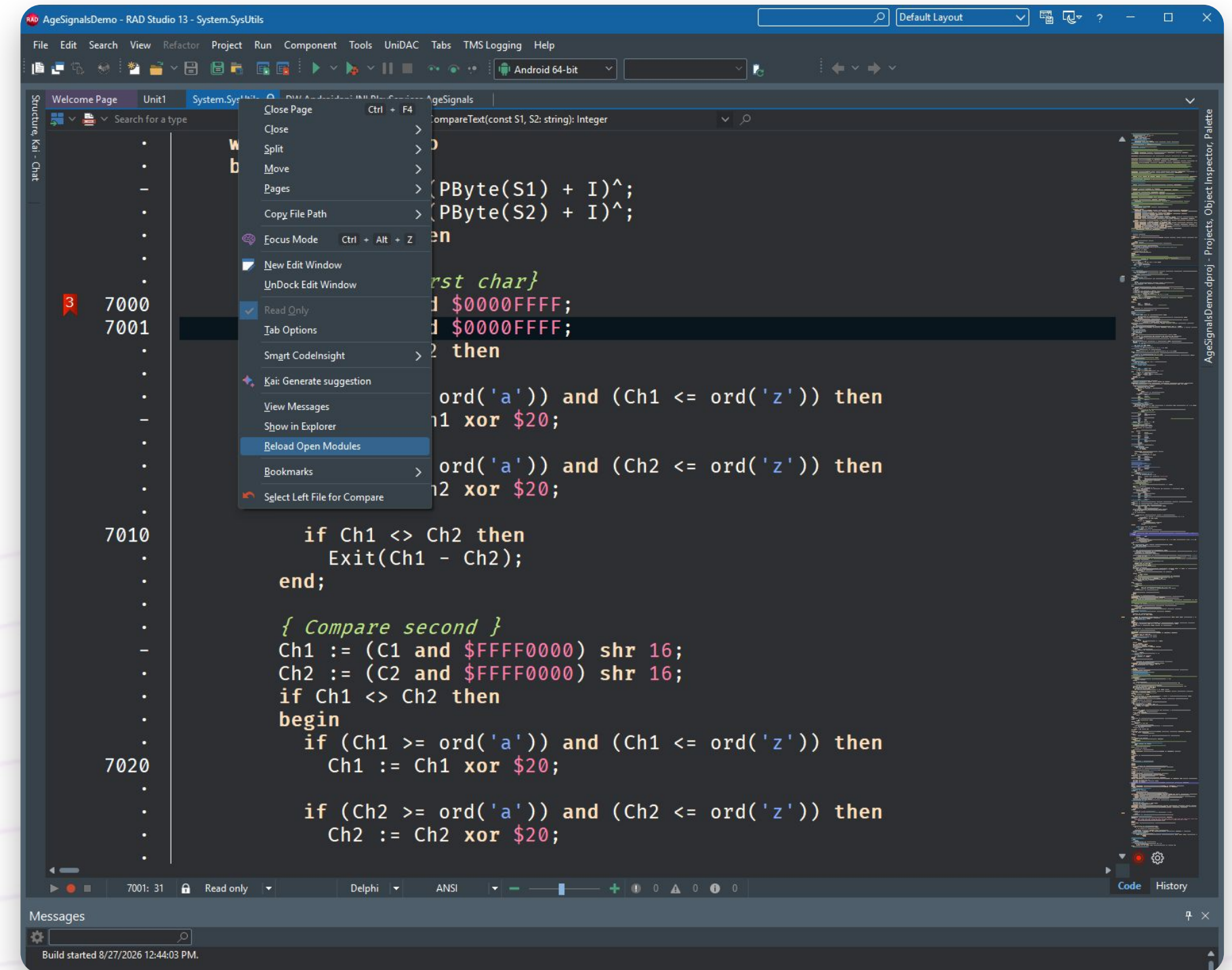
- Each language now has a dedicated options page
- Language-specific settings are easier to find
- The new organization removes the shared language dropdown



New CodeInsight language settings structure

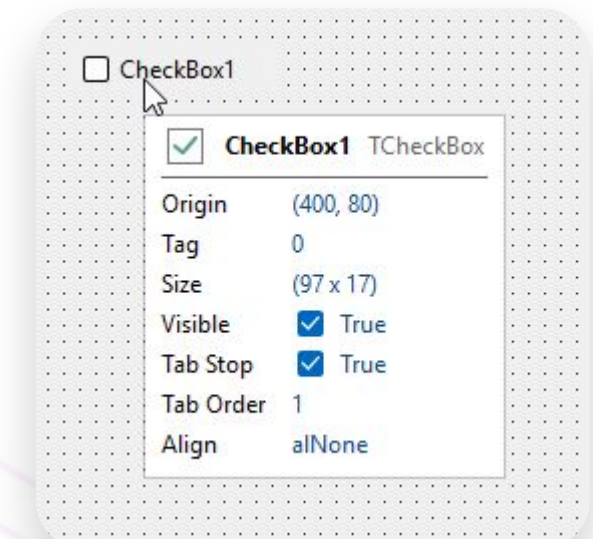
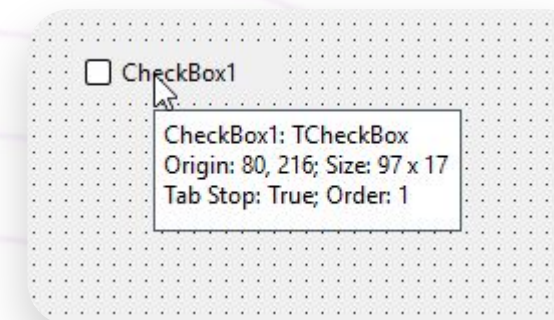
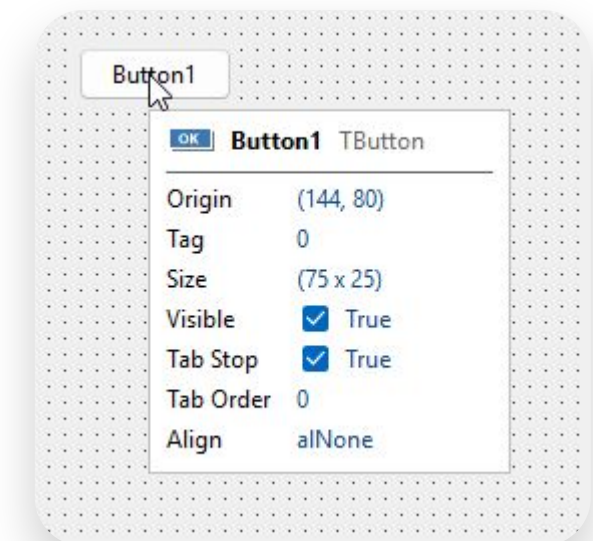
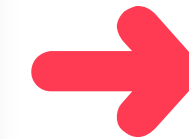
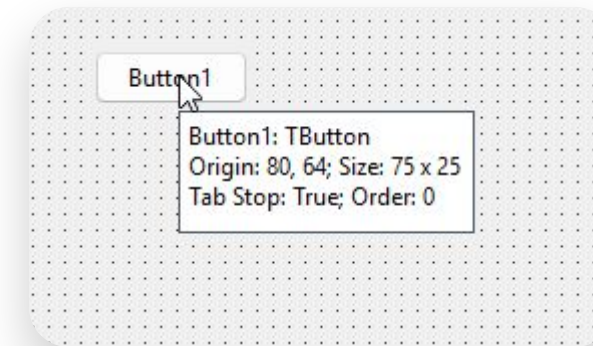
Everyday Editor Improvements

- Brace matching now handles strings and comments correctly
- Code insertion works reliably with “Linux” line endings
- C++ highlighting recognizes more modern syntax
- Reload Open Modules refreshes files changed by external tools



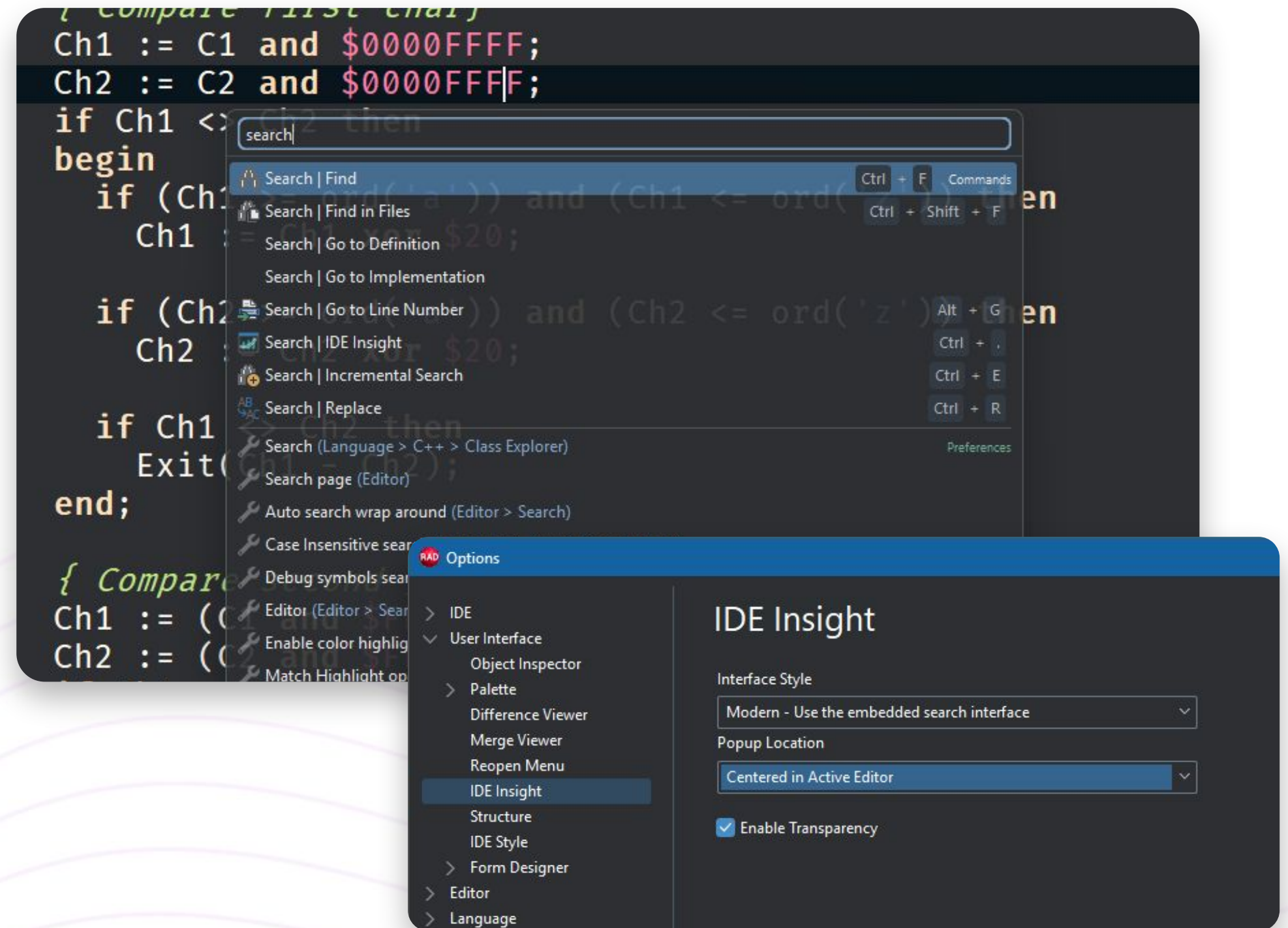
VCL Designer Hints and More

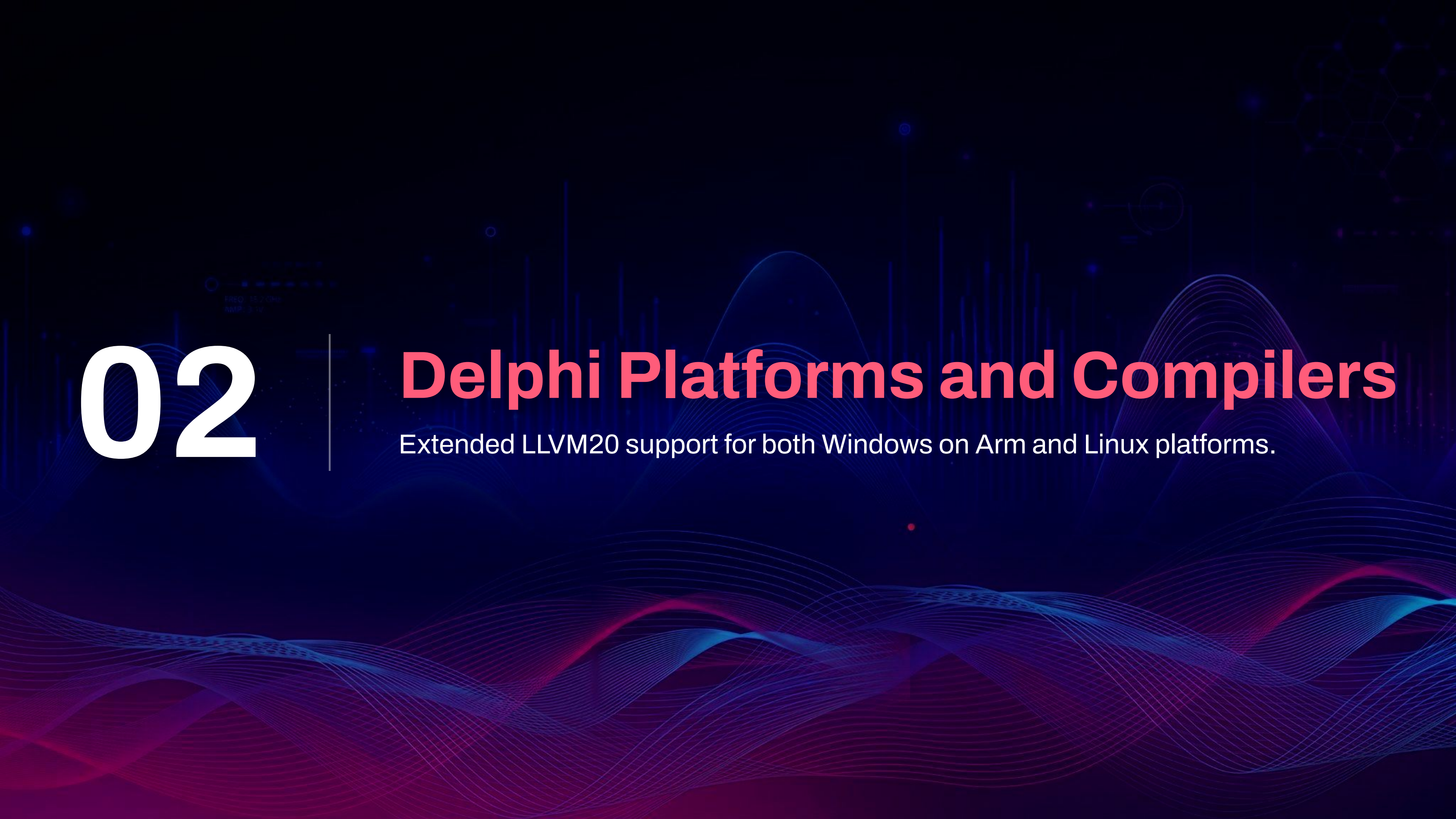
- Extended with more details, relevant for components and controls
- Proper formatting for readability
- Also in the Object Inspector, string properties are no longer limited to 2,000 characters
- Collection-item strings can use the multiline editor



Improvements Across the IDE

- Form inheritance works better in the High-DPI IDE
- New modern UI for menu shortcuts
- IDE Insight has improved settings, rendering, and feedback





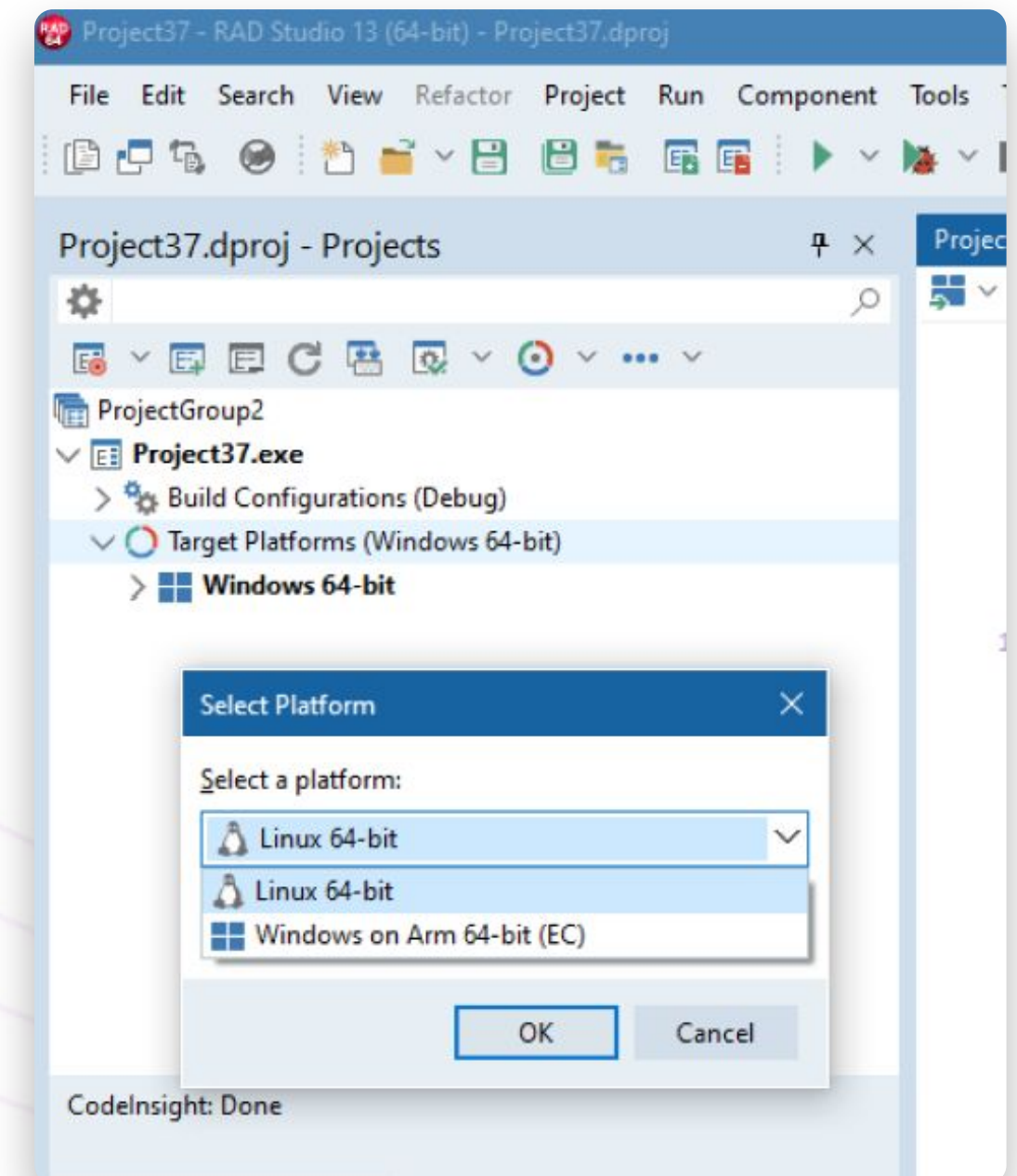
02

Delphi Platforms and Compilers

Extended LLVM20 support for both Windows on Arm and Linux platforms.

Delphi Linux Moves to LLVM 20

- In the Delphi Linux Intel-64 compiler, **LLVM 20.1.8** replaces a much older LLVM backend
- Both 32-bit and 64-bit compiler executables are available
- Linux Intel 64-bit can be targeted from both 32-bit and 64-bit IDEs



Adding Linux target in the 64-bit IDE

A Modernized Linux Toolchain

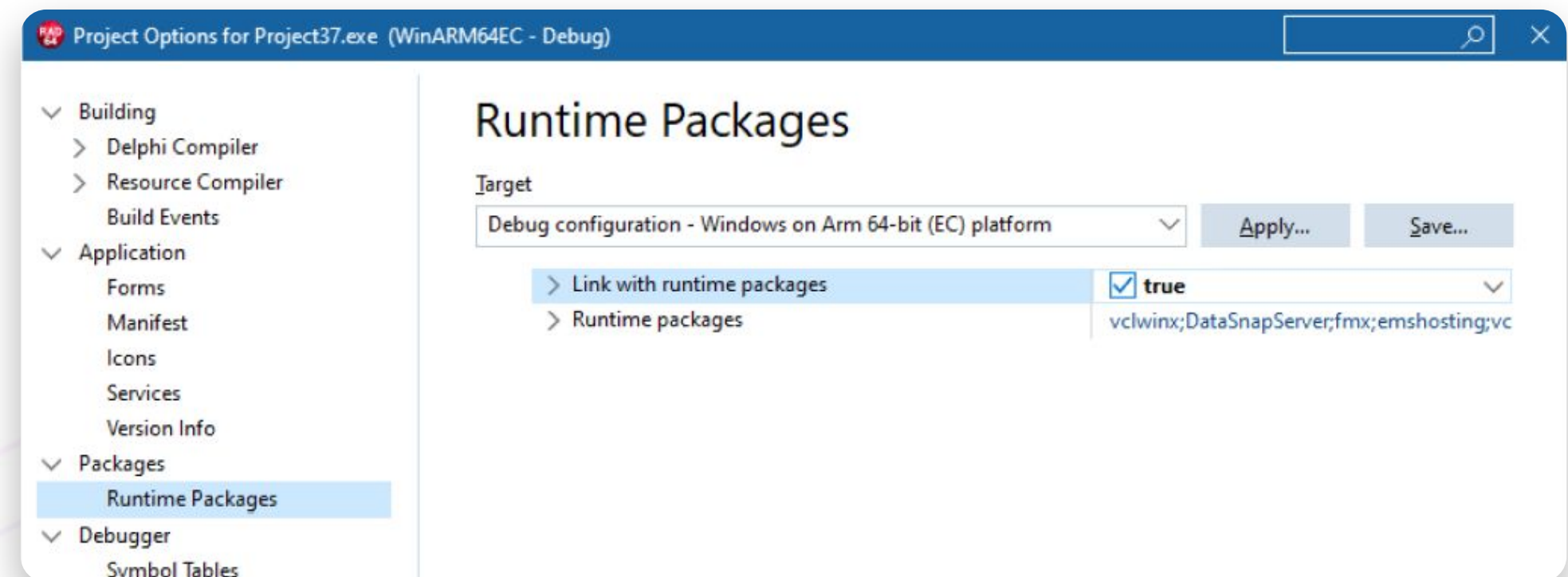
-  **LLVM LLD**
replaces the deprecated GNU gold linker
-  **DWARF 5**
replaces DWARF 2 debug information
-  **Compiler**
generates more complete debug metadata
-  **glibc 2.31**
establishes the new runtime baseline

Shares Delphi LLVM 20 compilers improvements with Arm64EC *(more on these later)*

-  LLVM optimizations improve Release application performance
-  rpmalloc is an optional memory manager
-  Parallel compilation to speed up development

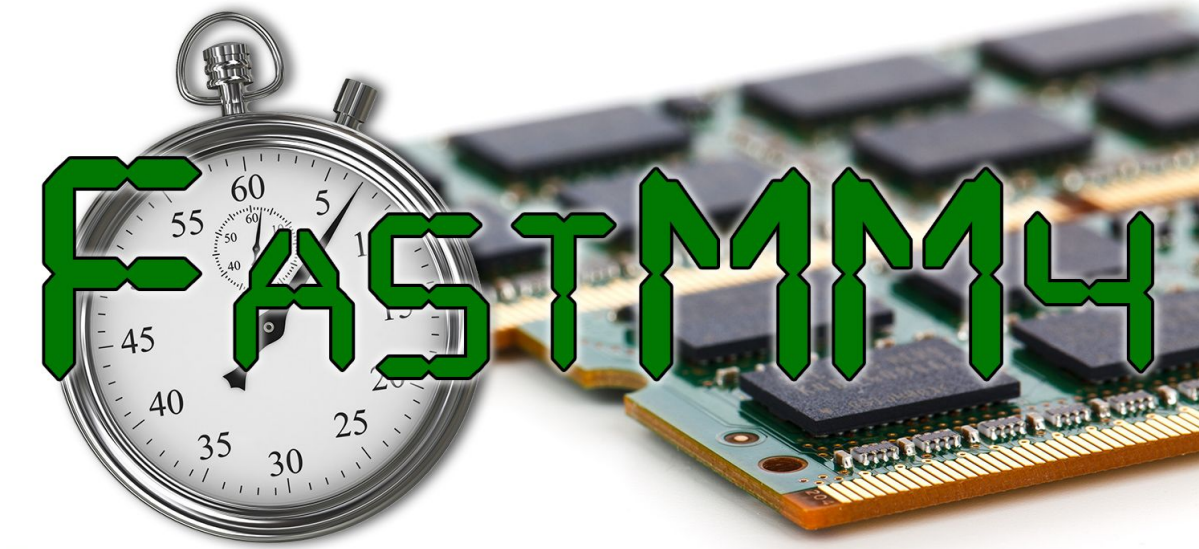
Runtime Packages for Arm64EC

- Create and consume dynamic packages
- Use Link with runtime packages on Windows on Arm
- Retain modular application and component architectures
- Arm64EC packages follow the platform's native ABI direction



Runtime Improvements on Arm

- **FastMM4** is now the default Delphi memory manager for Windows on Arm
- Reallocation-heavy operations, including string concatenation, perform much better
- Floating-point exception handling has been optimized
- Generic-heavy code builds faster
- Networking, resources, RTTI, and library
- Compatibility received additional quality work



Memory Management for Different Workloads

FastMM4

balanced default on Arm64EC, lower memory use, and leak reporting

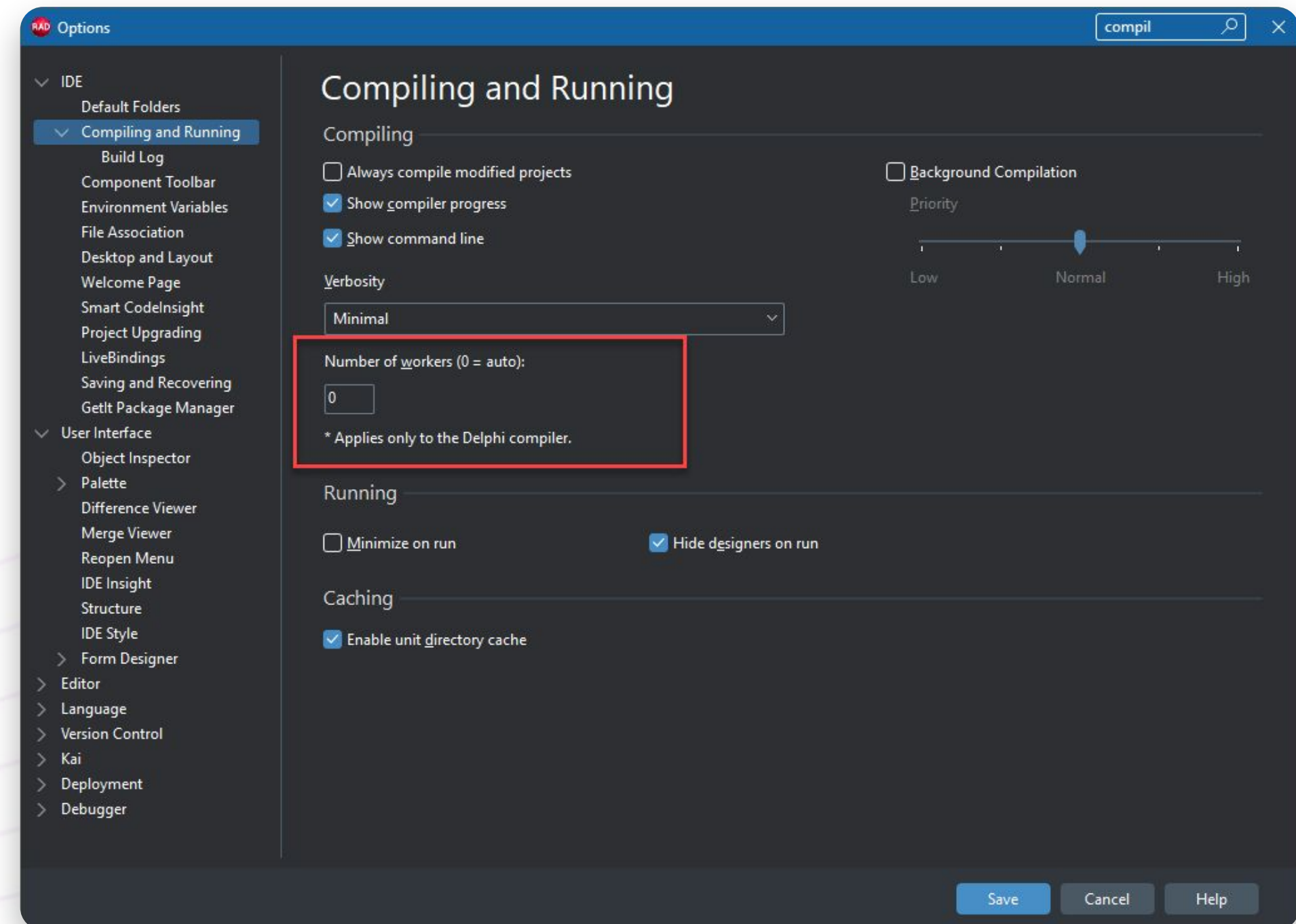
rpmalloc

optimized for heavy multithreaded allocation, optional on Arm64EC and Linux Intel 64-bit

The tradeoff is higher memory use and no equivalent built-in leak report
(ReportMemoryLeaksOnShutdown has no effect)

Parallel Code Generation Reduces Build Times

- For Delphi **LLVM 20** compilers **only**
- Process multiple Delphi units concurrently
- **Enabled by default** in the 64-bit compiler executables (32-bit ones have memory constraints)
- Select the worker count automatically or configure it – use **1** (worker) to disable it
- Control the feature globally, per project, or from the command line



New “Number of workers” option

Parallel Builds: Current Results

Rebuilding all FireMonkey units - as a test:



2x

Faster units in Debug



14x

Faster in Release

Measured on a 10-core Intel Core i9-13900H system. Results vary with project structure, hardware, and configuration

LLVM 20 Produces Faster Applications



Windows on Arm

 LLVM IR optimizations for Delphi

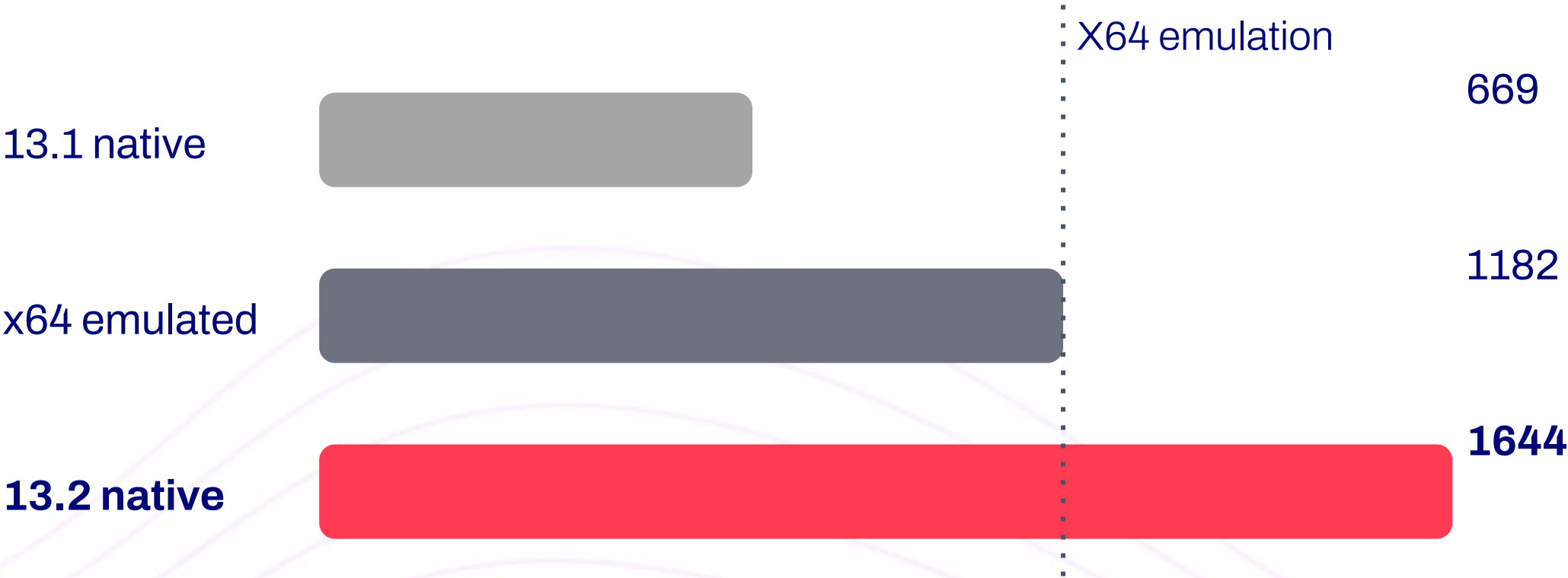
 Delphi exception semantics preserved

 **2x** to **4x** faster in internal tests

 Up to **7x** on specific workloads

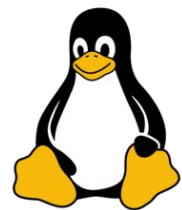
2.5x
Faster than 13.1

1.4x
faster than x64 emulation



SciMark 2.0 composite results (MFlops), Windows 11 on Arm. Up to 4.5x faster than 13.1 on individual benchmarks

LLVM 20 Produces Faster Applications



Linux x64

 Same hardware, same kernel

 Same Delphi source

 **3.8x to 5.1x**
faster than 13.1

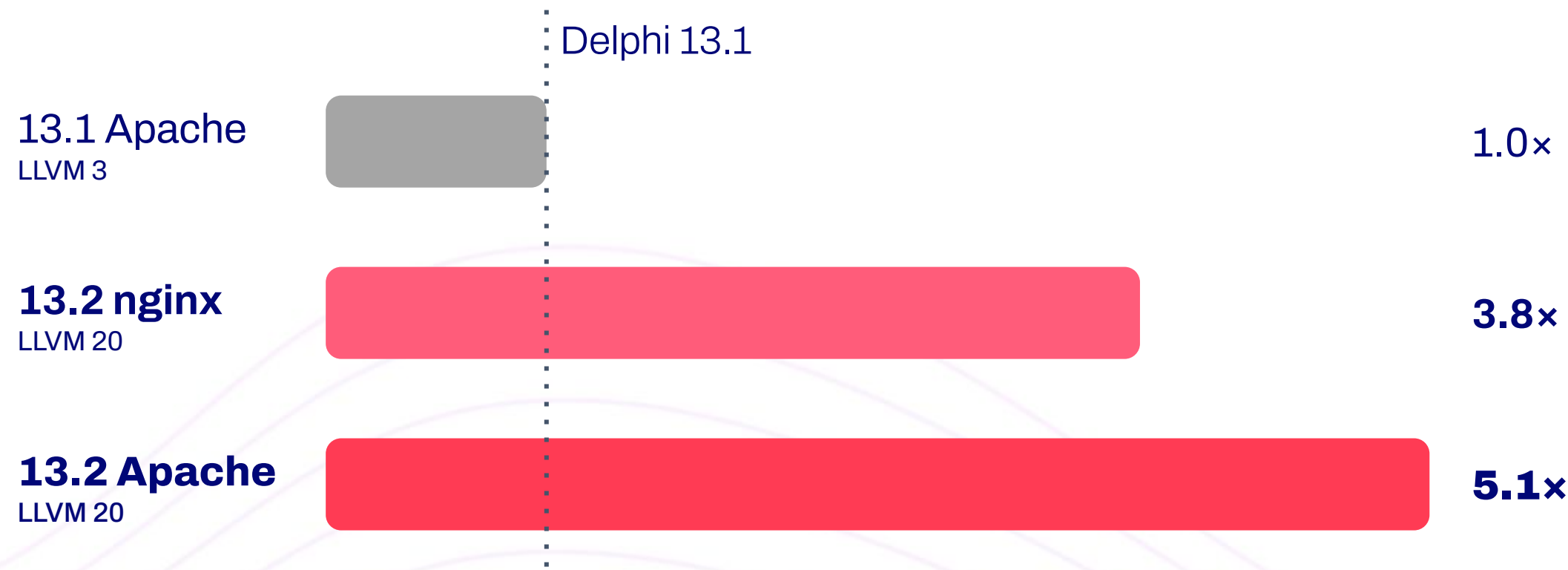
 **7.3 ms → 1.6 ms**
single-request latency

4.5x

Lower latency
7.3 ms → 1.6 ms

5.1x

Faster than 13.1 on Apache



Same fill+sum kernel. Bars: average speed vs 13.1. Latency numbers at concurrency 1. Compiler on vectorizable CPU work, not a WebBroker HTTP claim.
Hardware: 4 cores i5-13500 - 4GB RAM - Ubuntu 22

Faster Linux Apps in 13.2





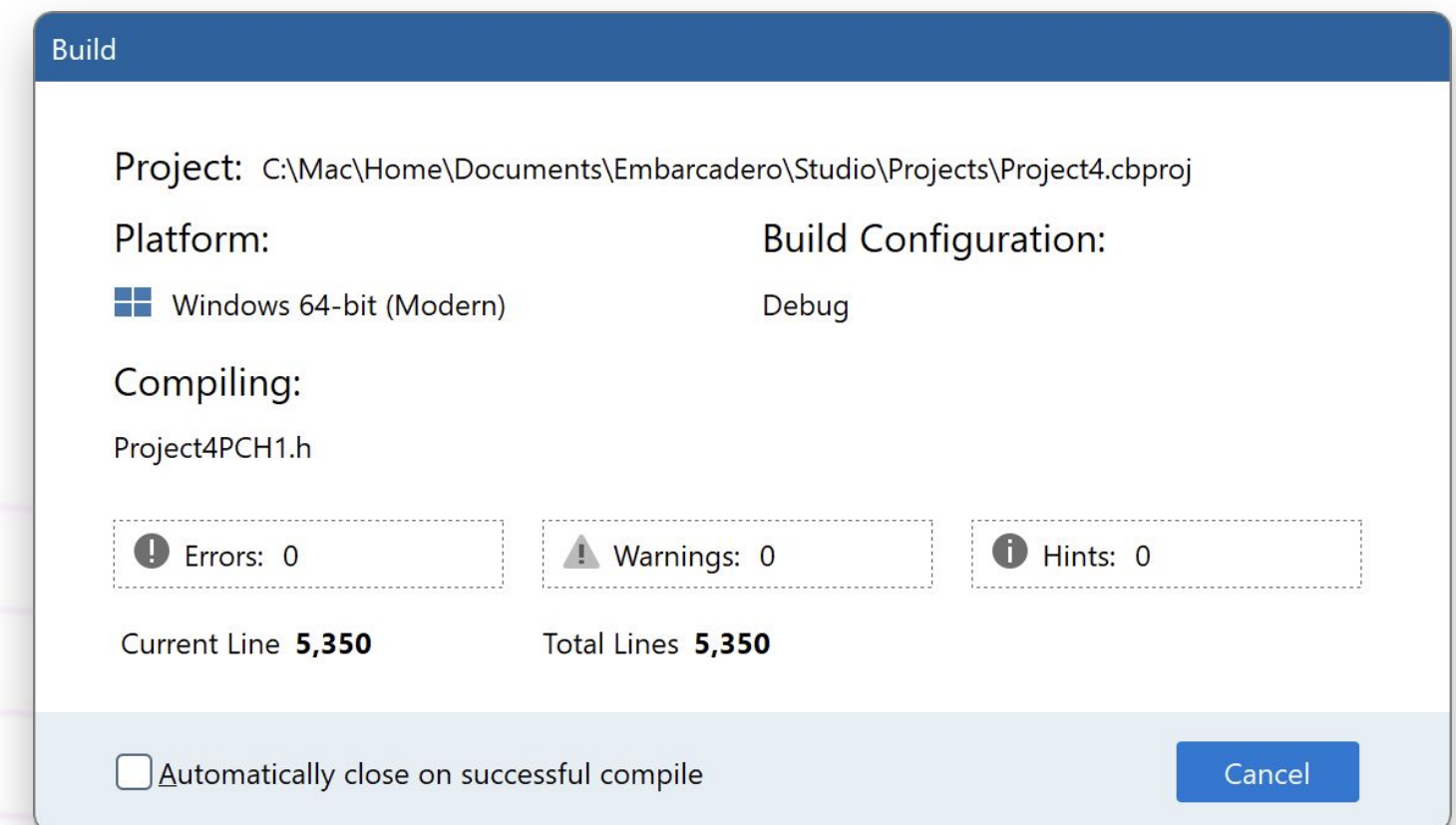
03

C++ Builder and Debugging

Higher quality build target with major 64-bit debugger updates

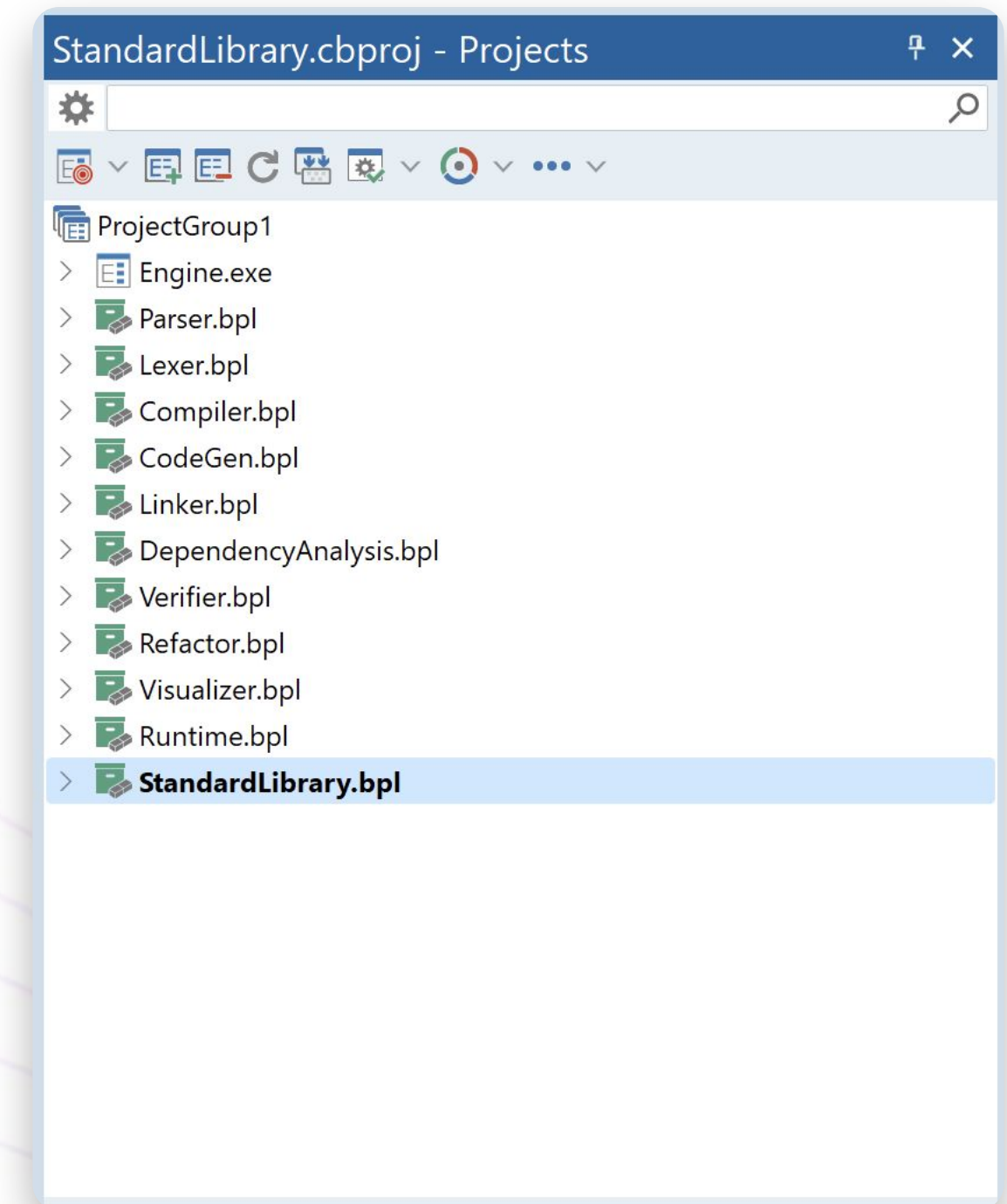
The Win64x Compiler Is Now In Process

- Run the Clang-based Win64x compiler **inside the 64-bit IDE**
- See accurate file and line progress
- Receive structured diagnostics in the Messages view
- Cancel builds promptly while keeping the IDE responsive



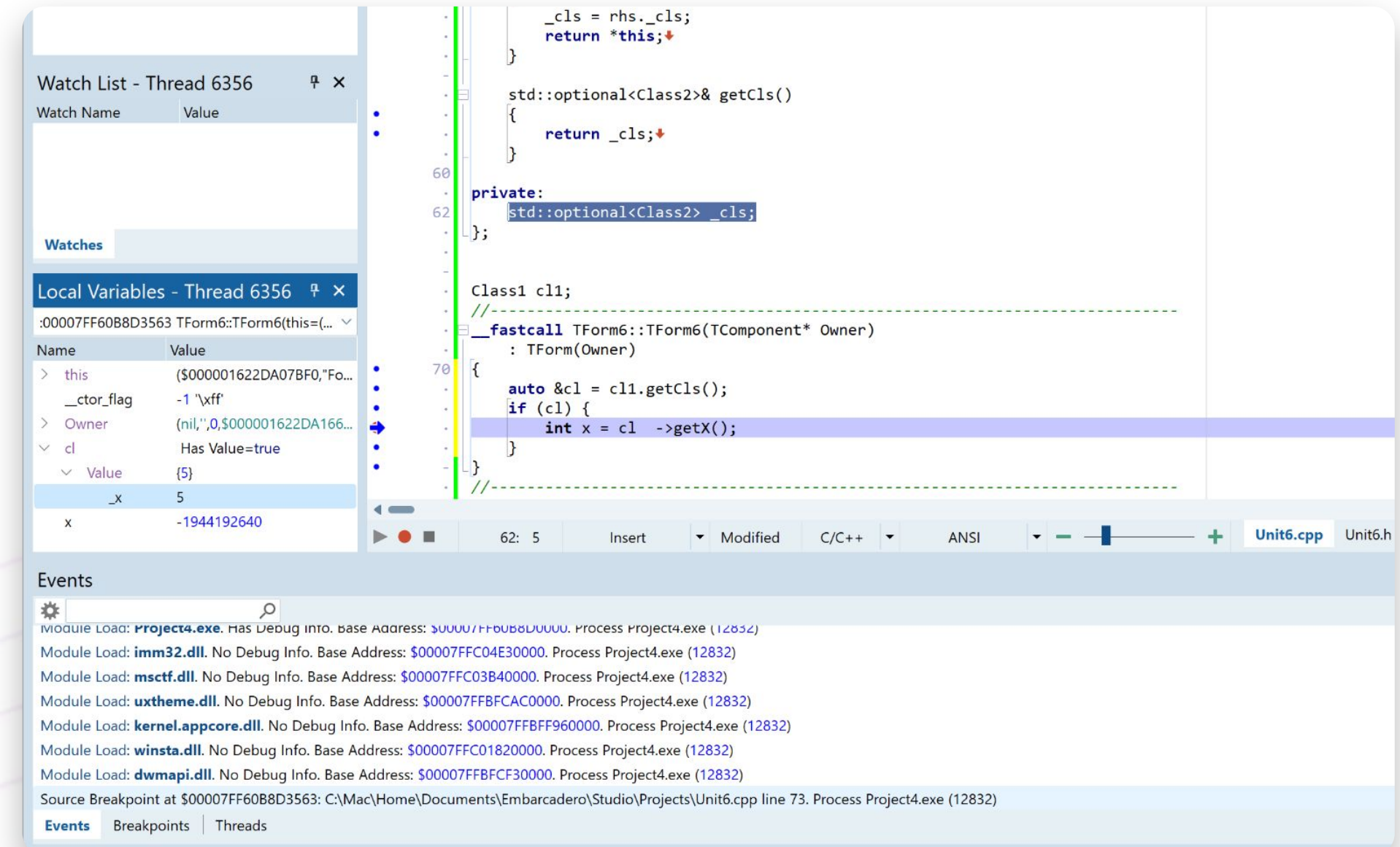
Packages and Project Reliability

- Better design-time 64-bit package support
- Improved packages containing Delphi and C++ units
Both are big rework of package support, bringing it more on par with previous compilers
- Automatic package dependency information generation
- Fewer unnecessary rebuilds after Project Options changes
- Fixes for resources, DFM paths, icons, and invalid property definitions



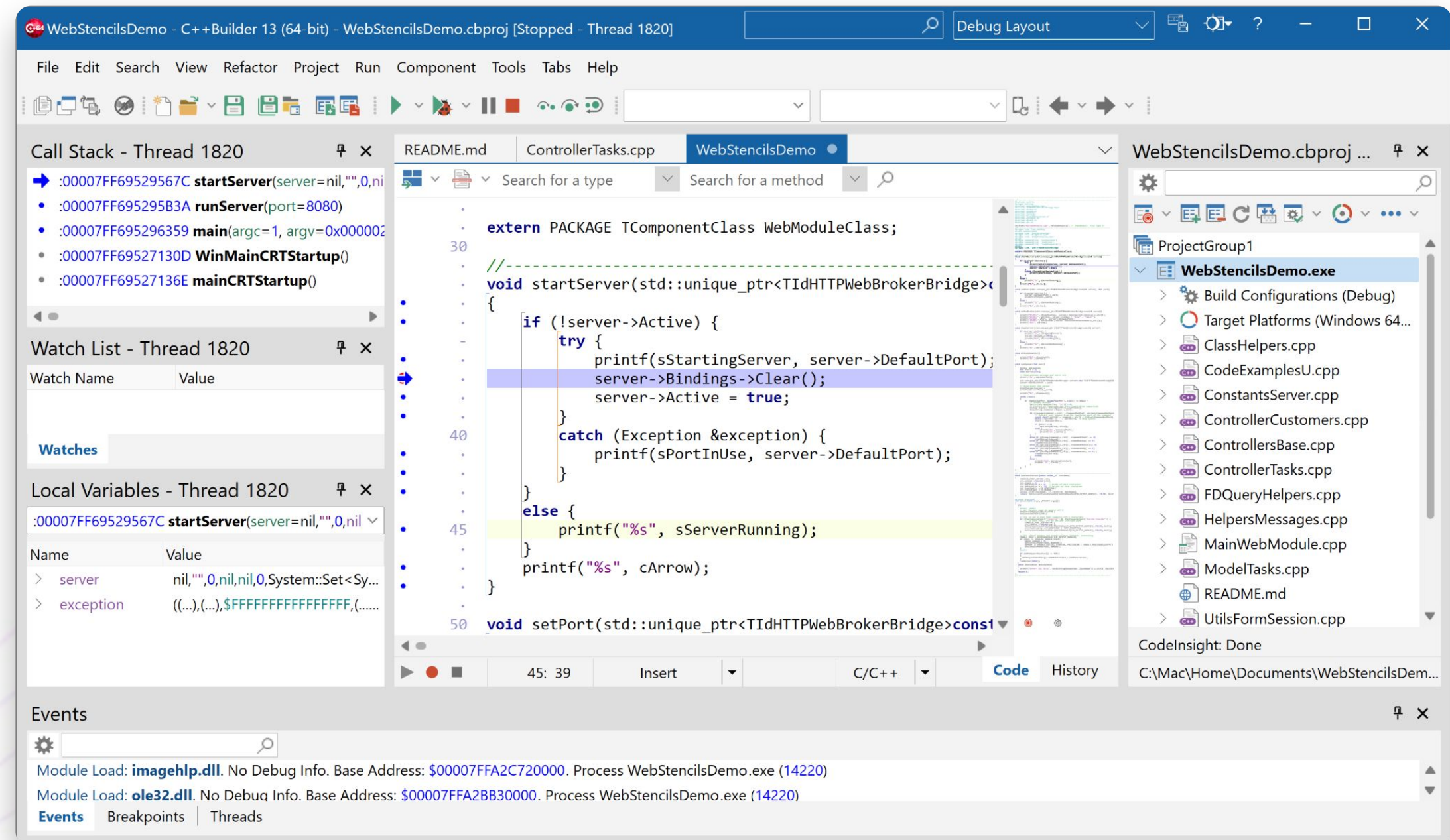
Inspect Runtime Data More Reliably via LLDB

- Improved **std::optional**, **std::any**, **std::variant**, and **std::tuple** visualization
- Better handling of STL iterators
- Better Delphi 64-bit display of Variants, arrays, and indexed properties
- More reliable expression evaluation and execution-point behavior



More efficient LLDB Debugging on Large Projects

- Choose which modules should have symbol tables loaded
- Read symbol information more efficiently
- Reduce unnecessary thread-information collection
- Improve stability of the IDE in such scenarios
- Pass user-defined environment variables correctly to the debugged process





04

VCL and FireMonkey

New VCL components and enhancements to the FireMonkey Style Designer

Two New VCL Navigation Controls

Continuing VCL UI controls modernization work

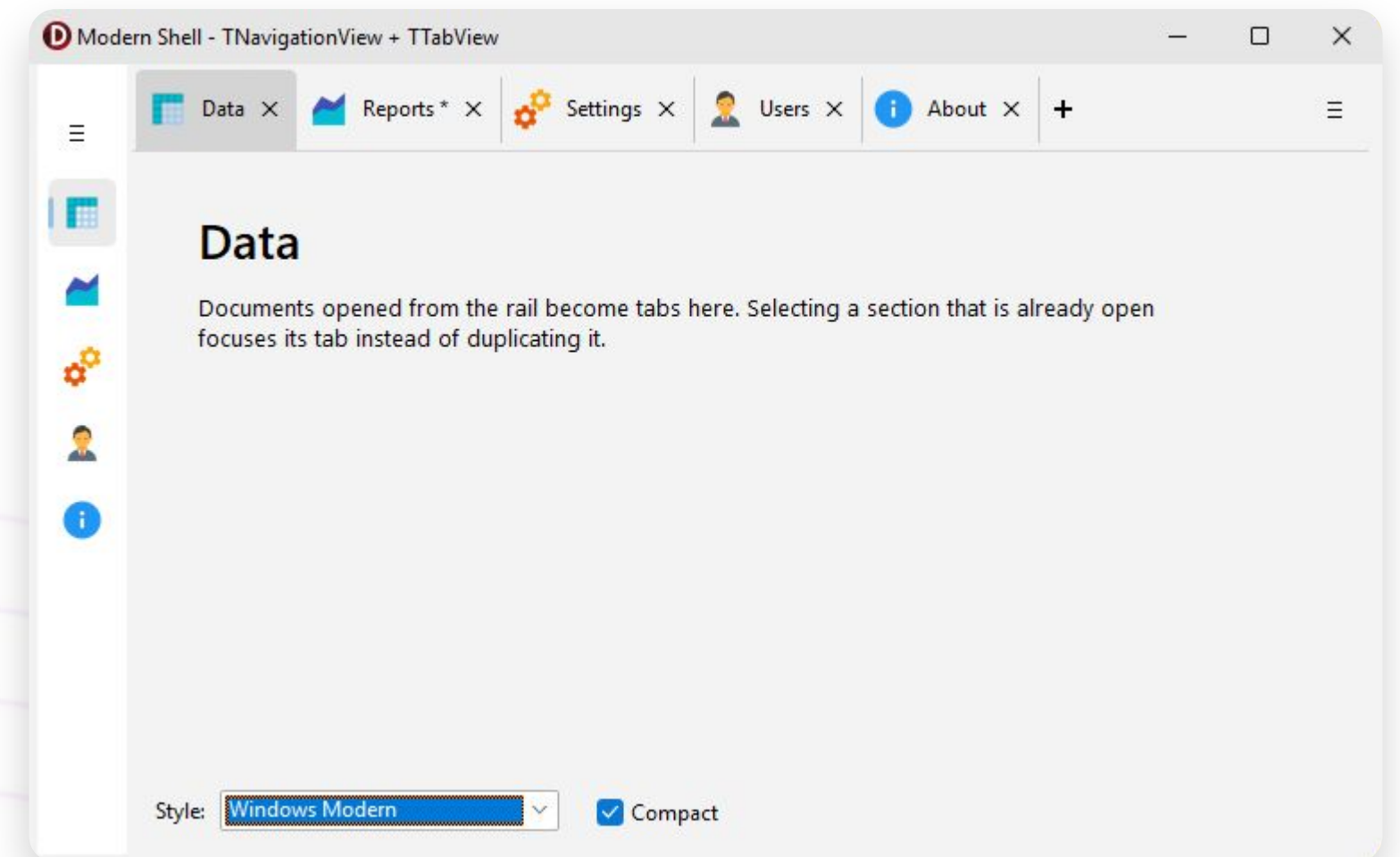
- Inspired by modern UI trends on desktop
- Taking into account Microsoft Win UI 3 controls

New in 13.2

- **TabView**: horizontal document and view navigation
- **NavigationView**: normal and compact vertical navigation

Both work with existing forms, frames, different panels, and other controls

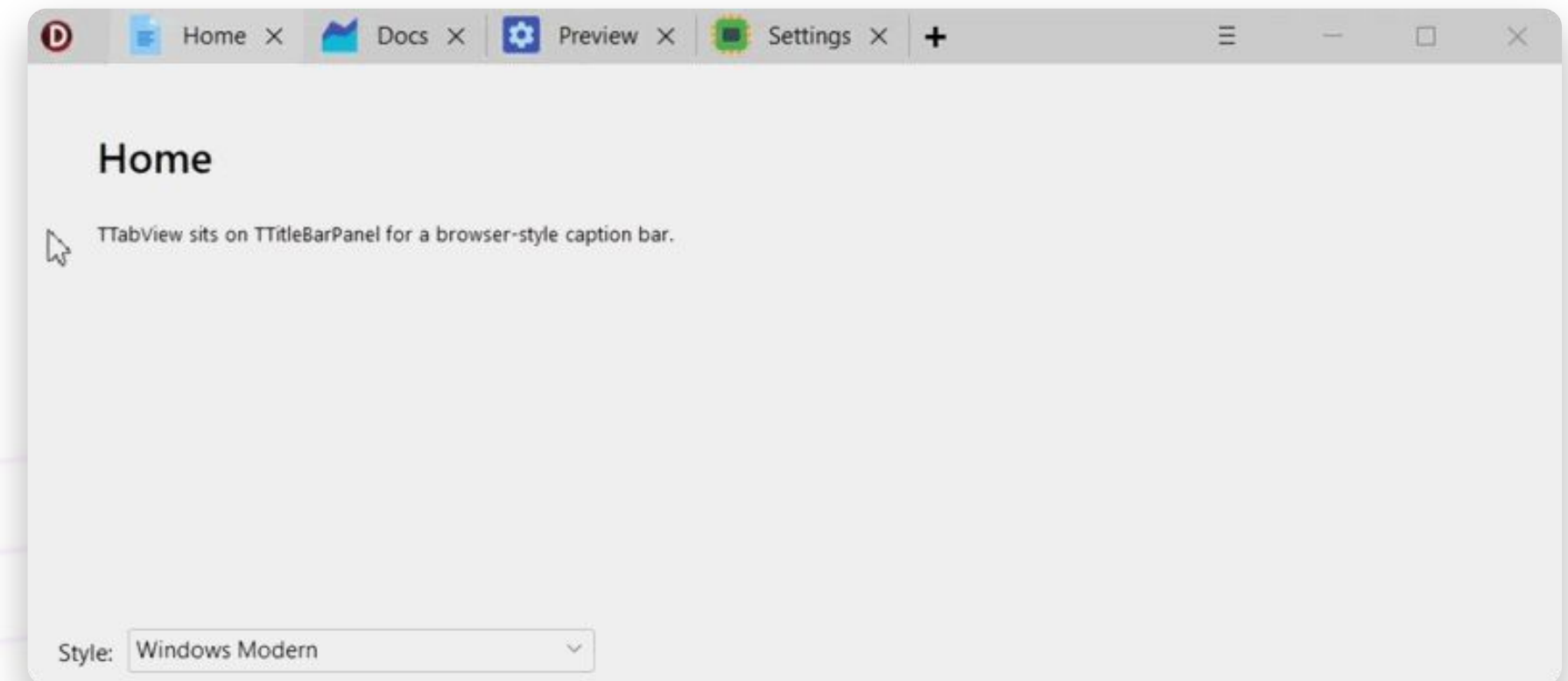
You can use them independently or together



Both TNavigationView and TTabView used together

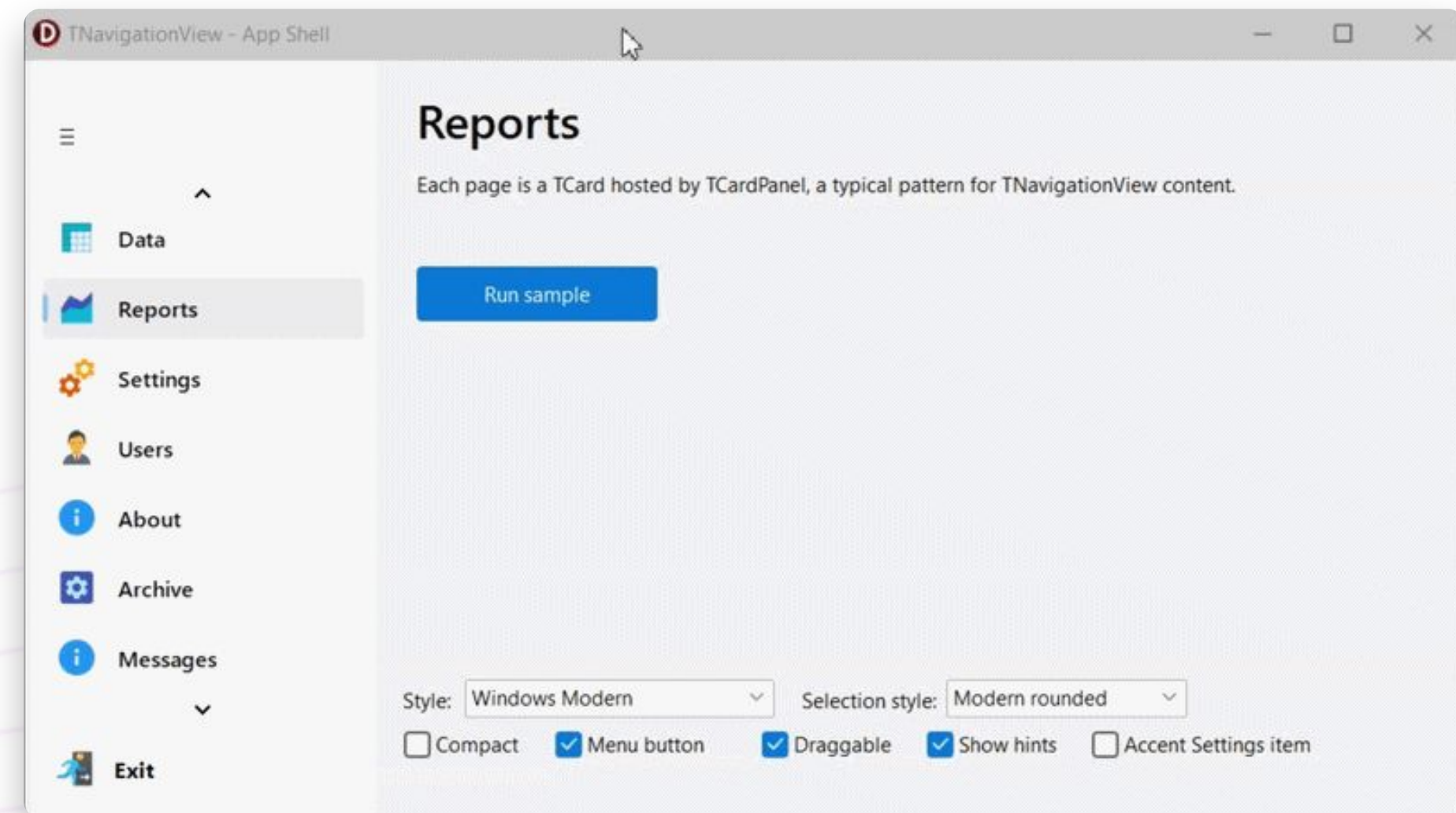
TTabView

- Associate tabs navigation with any control (panels, forms, etc)
- Tabs can have images, multiline text, and close buttons
- Includes buttons like new-tab, all-tabs, and scrolling
- You can place the control on a TitleBarPanel (as in this demo)



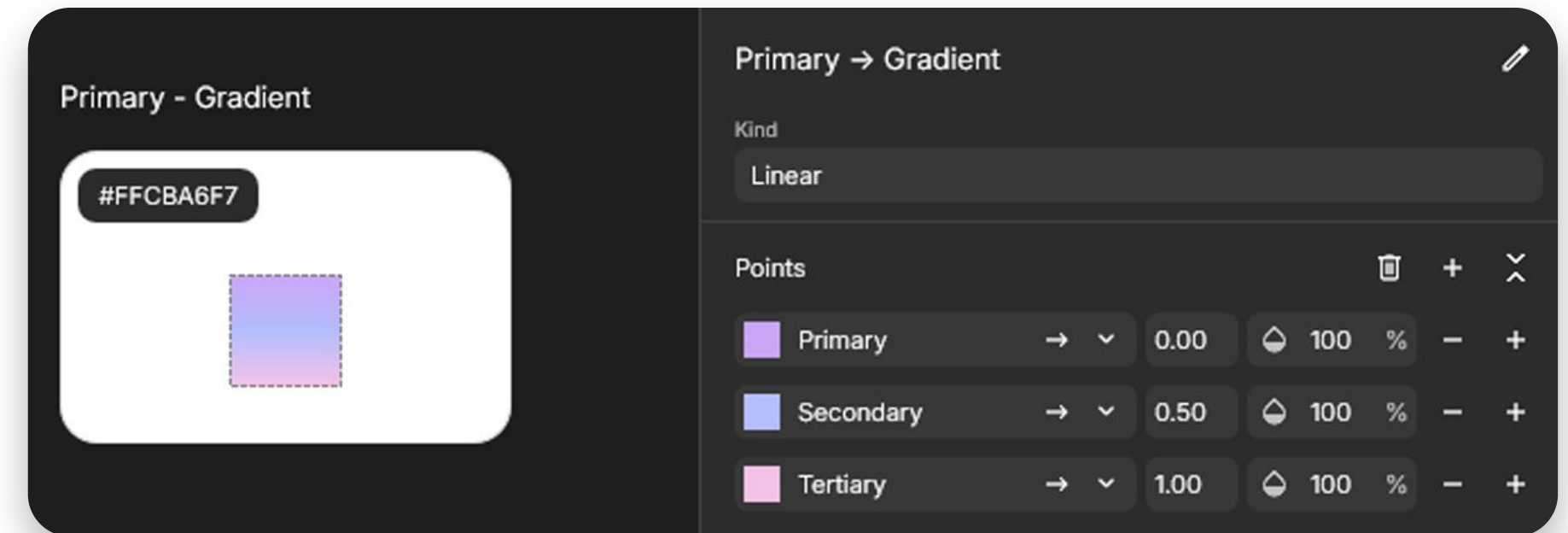
TNavigationView

- Modern side menu
- Switch between normal and compact view
- Entries can have images, multiline captions, hints
- Offers drag-and-drop reordering
- Works with built-in selection styles and VCL Styles
- Connect items to existing application pages and controls

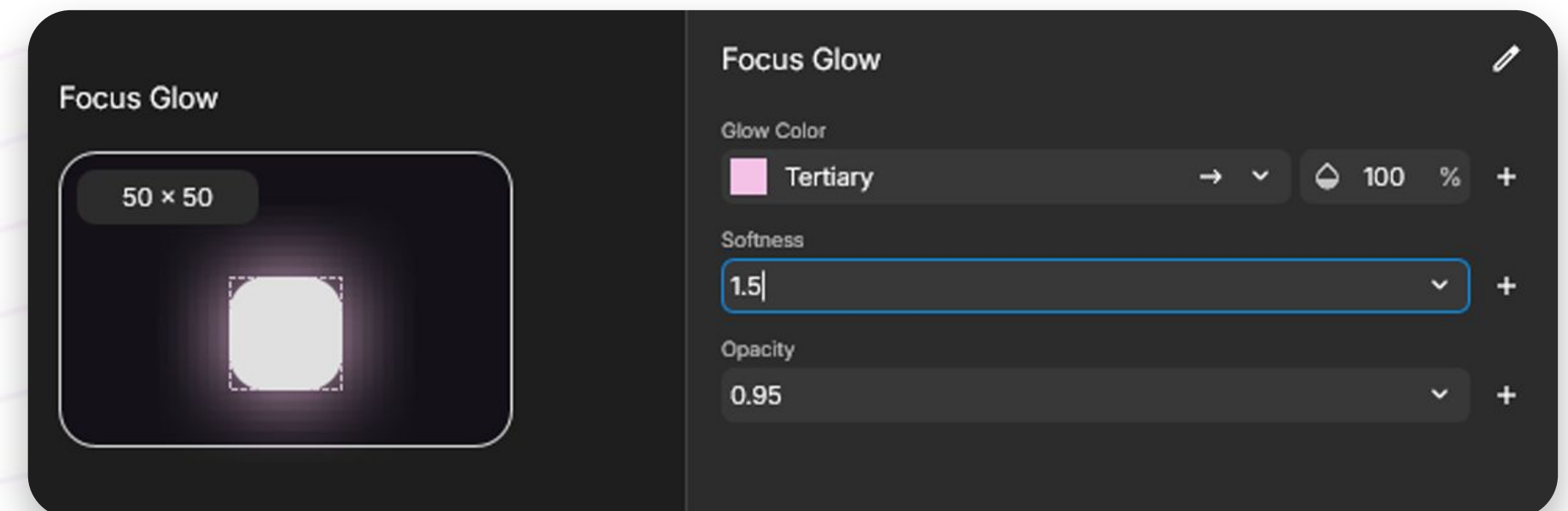


The FireMonkey Style Designer Grows Up

- Open the designer **directly from the IDE**
- **Import VSF files** from the Bitmap Style Designer
- **Use gradients**, multiple effects, glow, Circle, and Ellipse
- Control text layout and effects in more detail



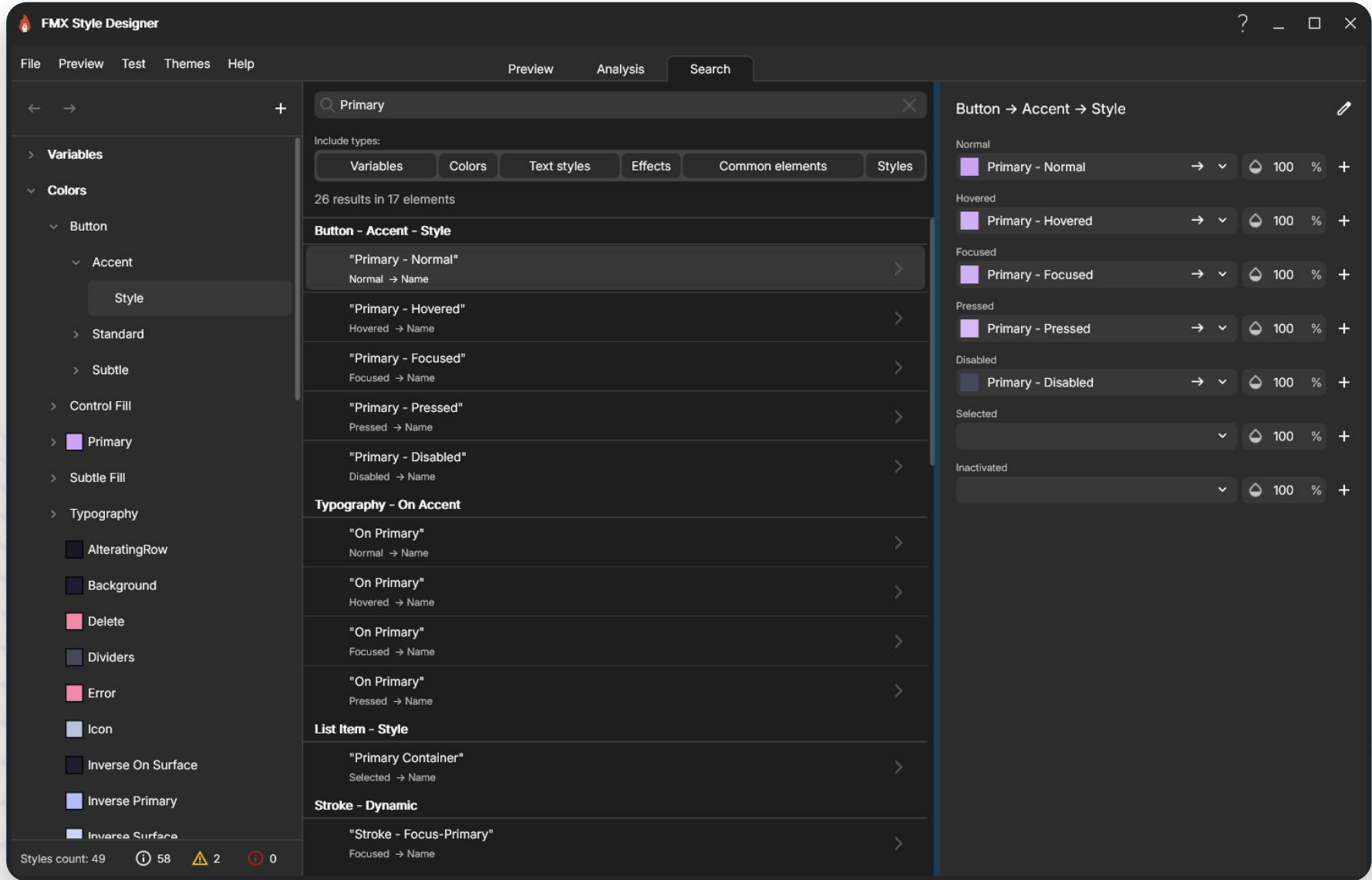
Multi-point Linear and Circular Gradients



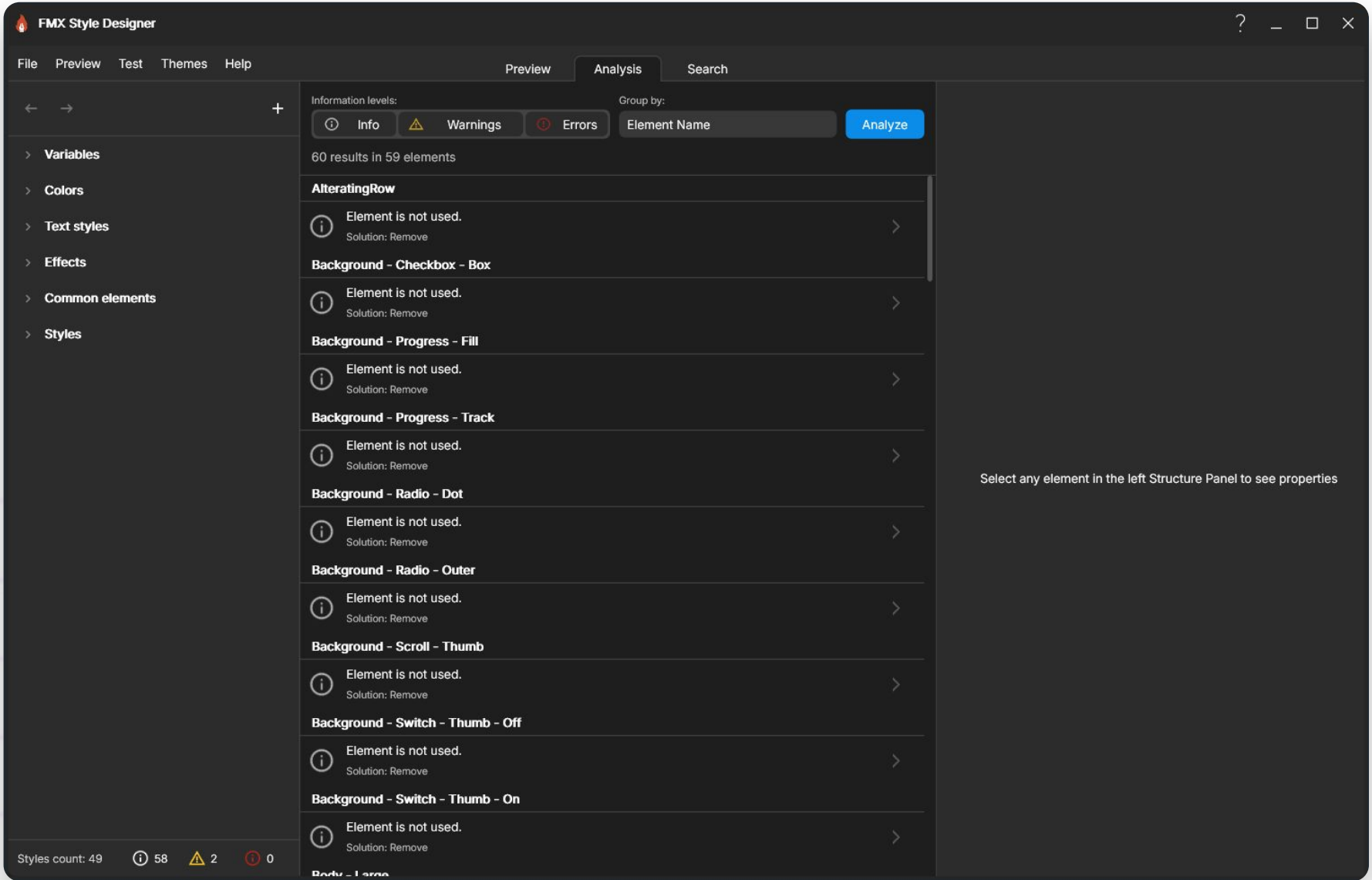
Glow Effect

The FireMonkey Style Designer Grows Up

Search and analyze larger styles



Fuzzy Search



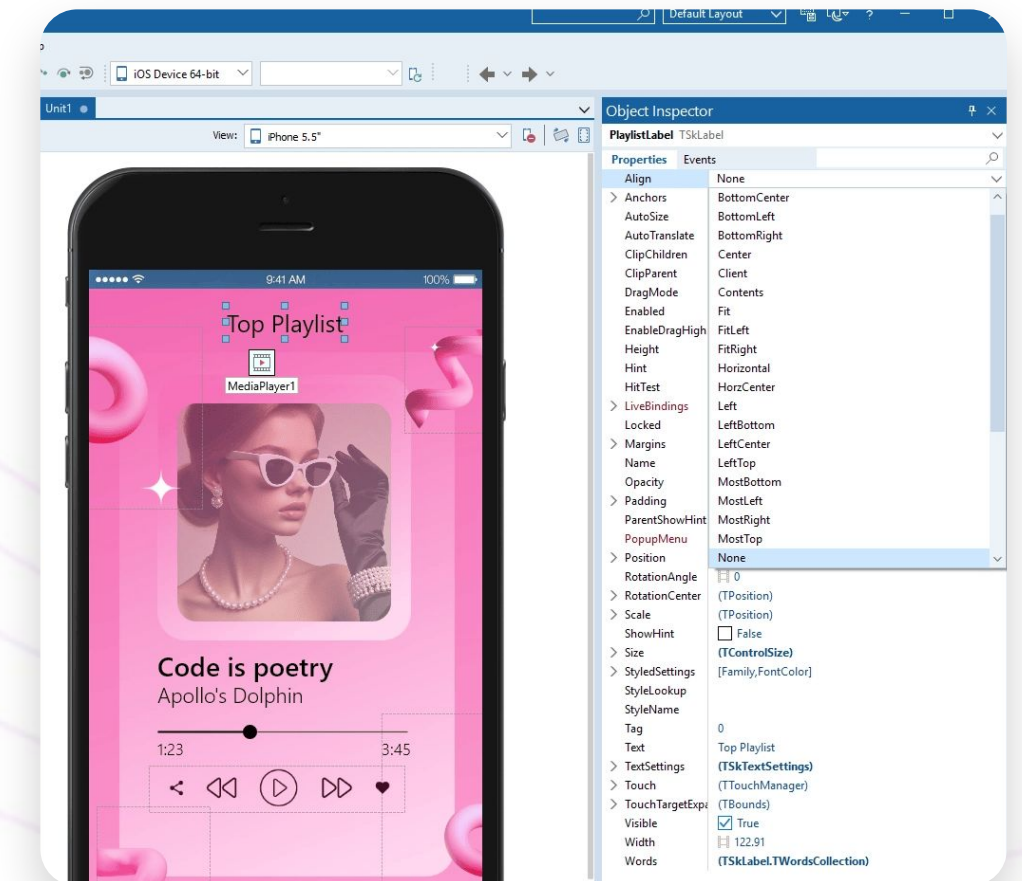
Project Analysis (Info, Warnings & Errors)

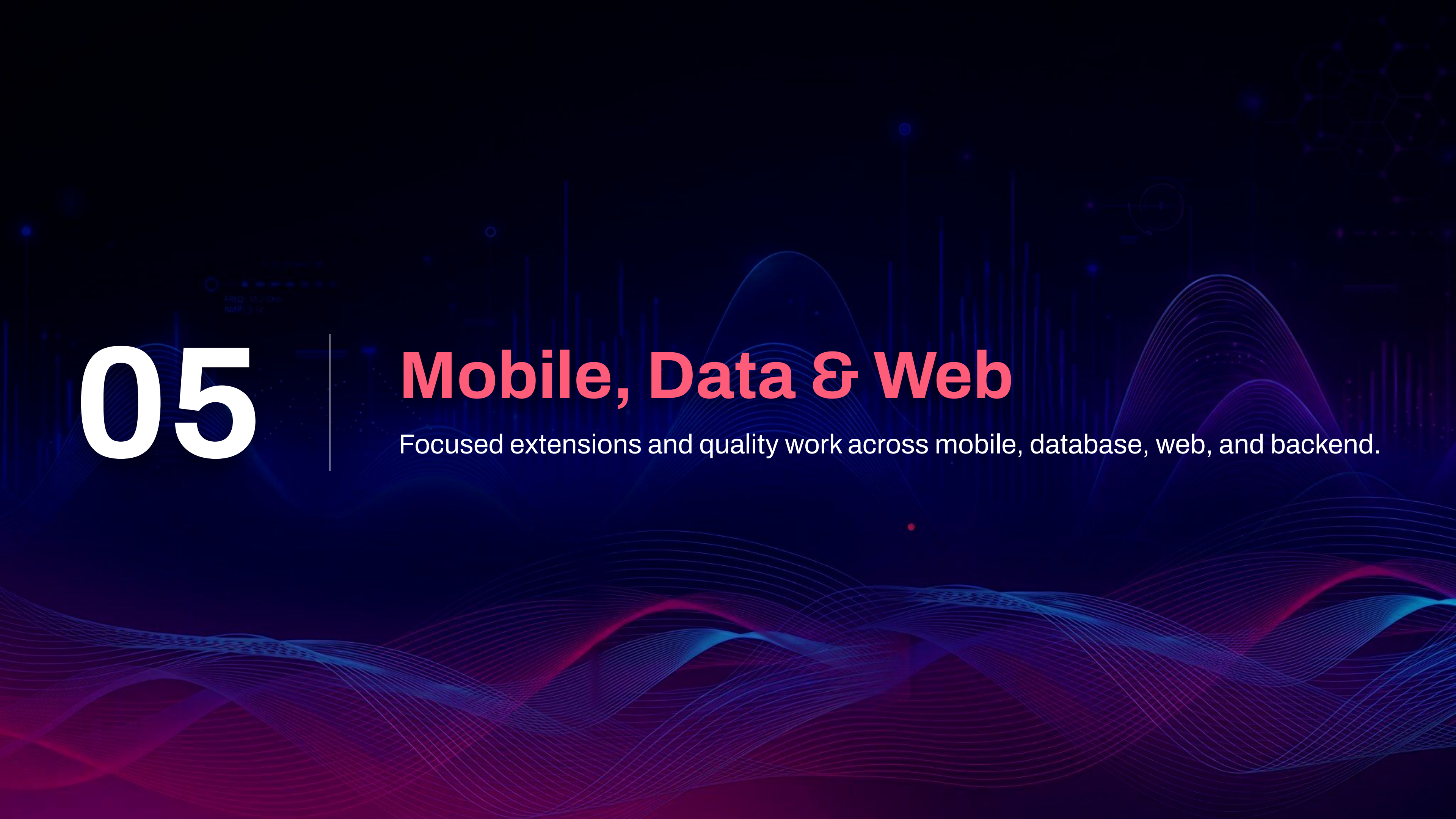
FireMonkey Quality and Rendering

- Skia4Delphi updated to version 7.3.0
- Includes a more recent version of the underlying Skia library (Chrome/m151, from August 2026)
- Requires a newer version of Vulkan
- Improved compatibility across Windows and Linux
- Better animation responsiveness on Windows
- High-DPI support for FireMonkey style-viewer tools

SKIA

FOR  DELPHI





05

Mobile, Data & Web

Focused extensions and quality work across mobile, database, web, and backend.

Updated Mobile Platform Support



Platform Updates

- Android minimum platform level updated to **API 24**
- Updated Google Maps and Google AdMob libraries



Age Verification

- Added low-level access to **Age Verification APIs** on both Android and iOS
- Actual or coming legal requirements in some US states and in Brazil
- Android entitlement support for the Age Signals service

FireDAC Database Updates

We keep testing and updating the FireDAC drivers for new versions of all 20+ supported databases.



Support for PostgreSQL 18.4



Configurable Oracle CLOB prefetching



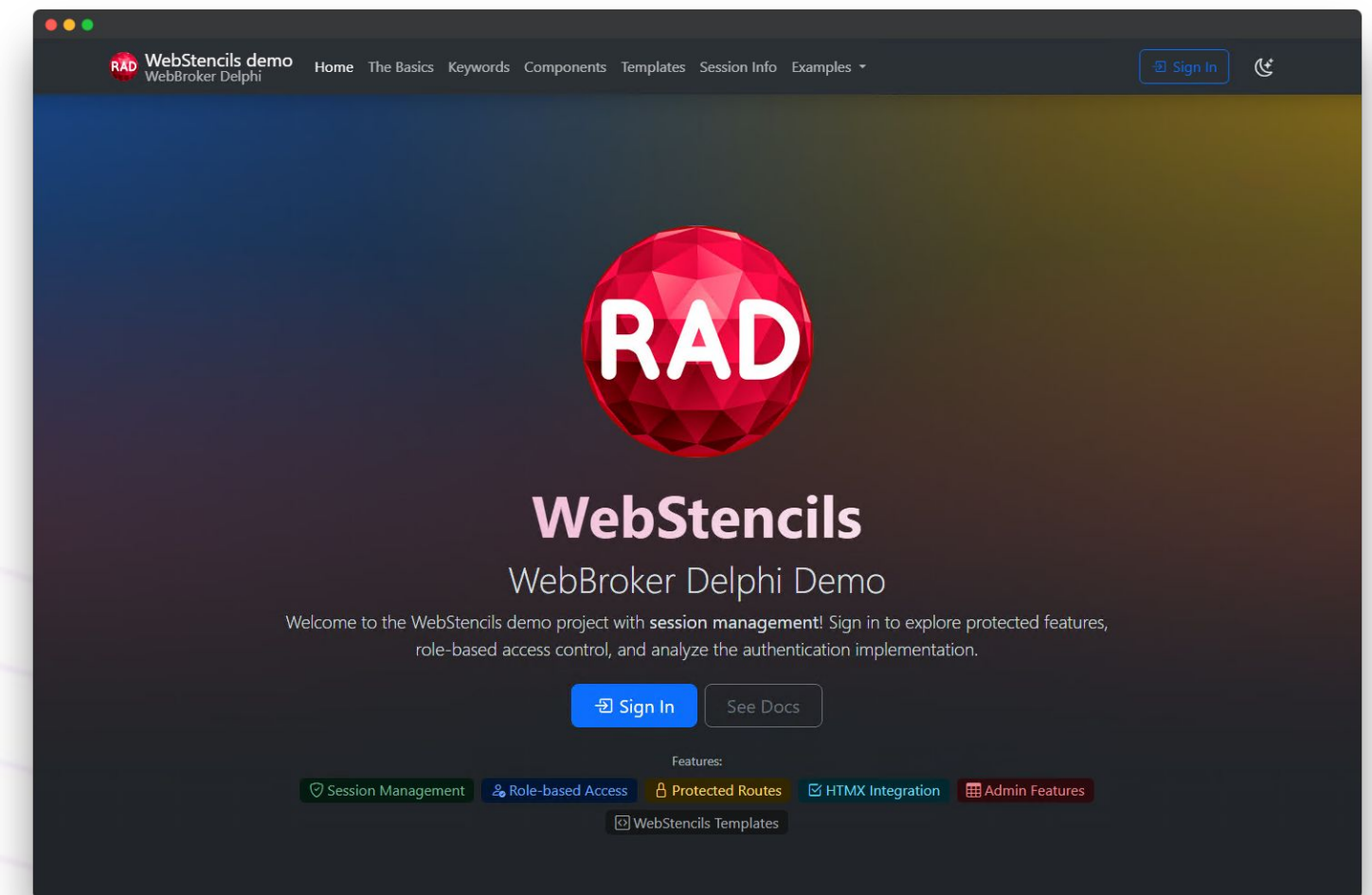
IBM Informix ODBC connectivity on Linux



More options for database applications deployed across platforms

Internet and Web Development

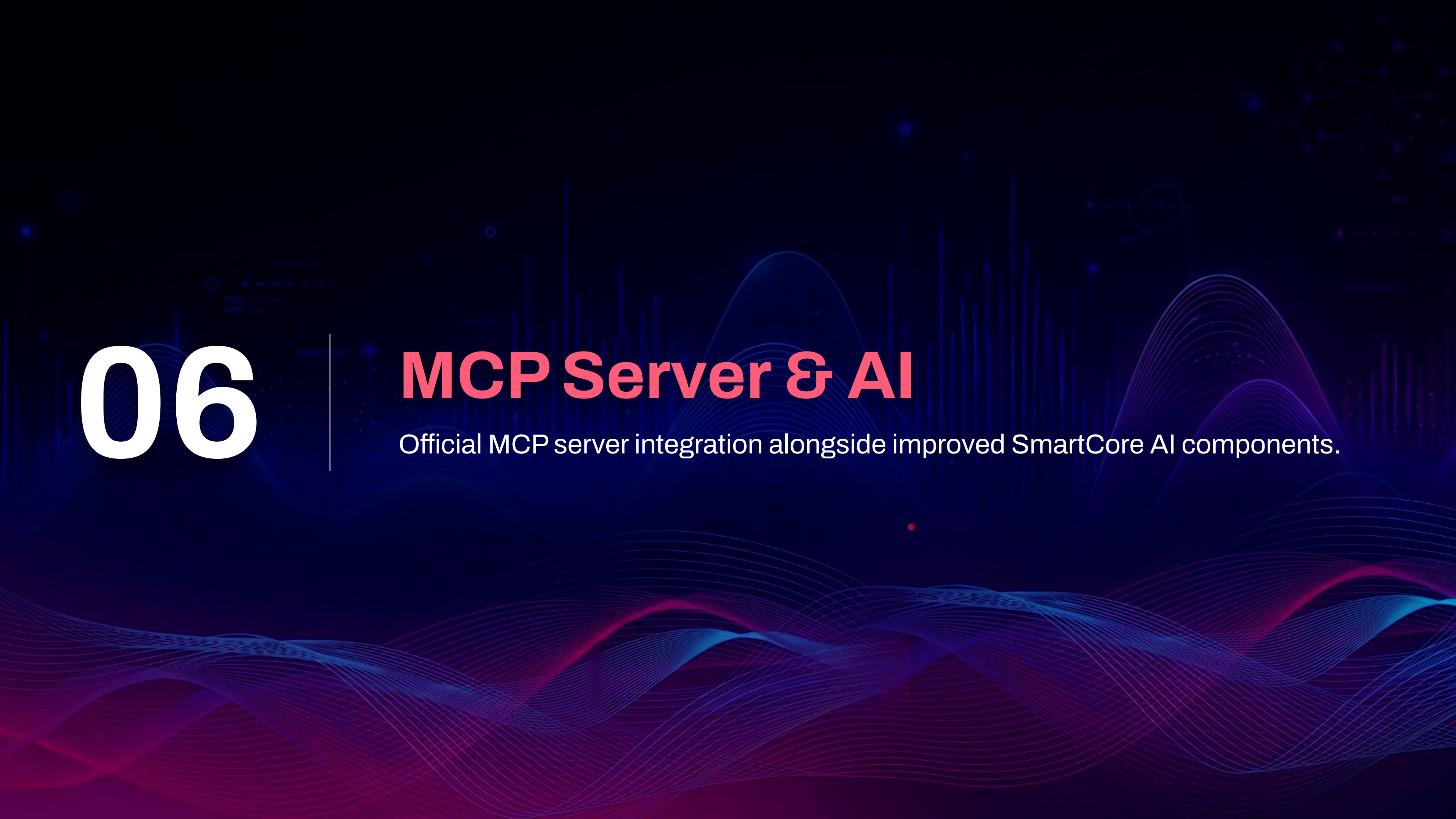
- OAuth 2 Authorization Code Flow with PKCE
Access to the native certificate context on Windows and Linux
- Added a specific socket timeout exception
- Improved REST Debugger errors and URL matching
- **WebStencils**
 - Explicit access to dataset fields
 - Improved path vs mask matching logic, when path is “/” or <empty string>



RAD Server Management

- **RAD Server Management Console**
 - High-DPI-aware UI
 - Sort users and groups by selected columns
- **Improved logging**
 - Log files accessible while in use





06

MCP Server & AI

Official MCP server integration alongside improved SmartCore AI components.

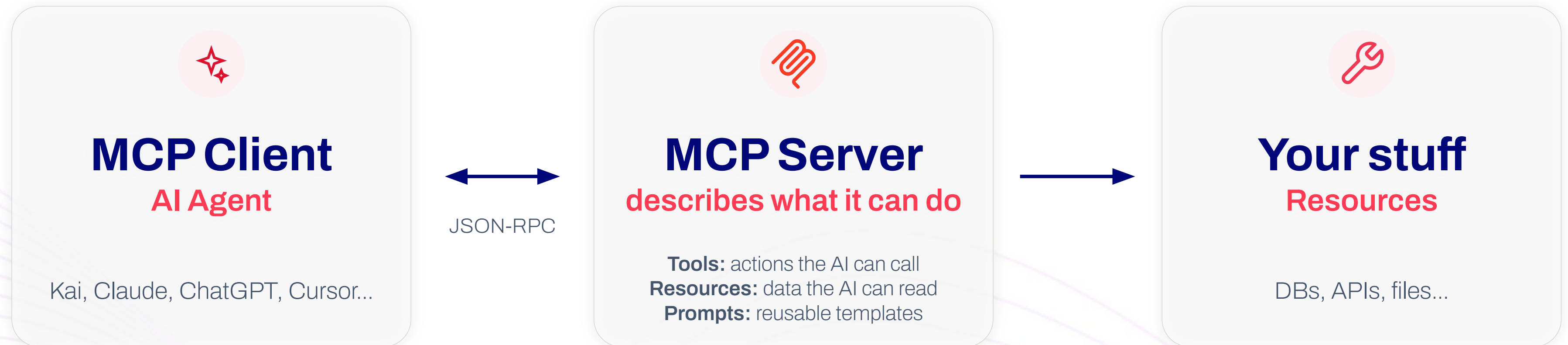
Updated AI Components for Delphi and C++

- **SmartCore AI Components helps you integrate AI in your apps**
- Significantly refactored SmartCore AI components
 - Access additional APIs and capabilities from supported AI engines
 - An updated foundation for AI-enabled applications
 - Same high end components, with reworked architecture
- Available (soon) in GetIt for Delphi and C++Builder



What is MCP?

Model Context Protocol is an open standard that lets AI assistants call tools and read data the same way, no matter which AI or which application is on the other end.

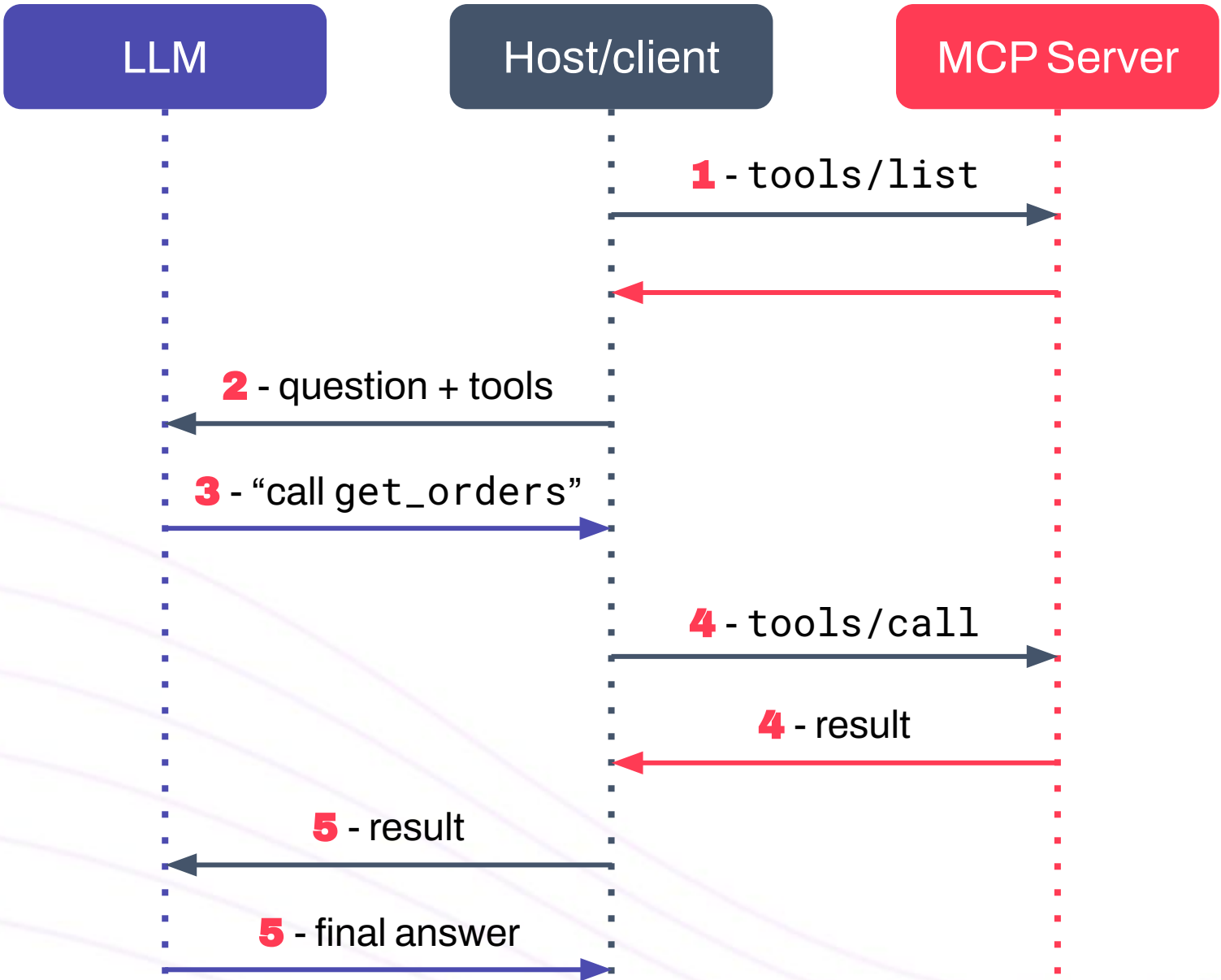


keep **tools** as the main idea, most people only need that one. **Tools = functions with a name, description and parameters**. The description is what the LLM reads to decide when to use it.

What is MCP?

What happens when you ask a question: *“How many orders did we ship last week?”*

- 1 On startup the client asks the server: **tools/list**.
Server answers with **get_orders(from, to, status)**
- 2 User asks the question. The host sends it to the LLM together with the tool list
- 3 The LLM doesn't know the answer, but sees a tool that fits. It replies: *“call **get_orders** with these arguments”*
- 4 The client sends **tools/call** to the server. The server runs real code and returns the data
- 5 The result goes back to the LLM, which writes the final answer:
“You shipped 142 orders”



Build MCP Solutions with MCPConnect

MCPConnect is bundled as an **additional GetIt download**



Why MCPConnect

- Build MCP clients and servers in Delphi and C++
- Connect applications, data, and services with MCP-compatible AI tools
- Use a ready-made library already tested by the community and supported by Embarcadero, instead of implementing the protocol from scratch

```
type
  THelpDeskService = class
  public
    [McpTool('doclist', 'List all the available documents')]
    function ListDocument(
      [McpParam('category', 'Document Category')] const ACategory: string
    ): TContentList;

    // This method is NOT exposed – it lacks the [McpTool] attribute
    procedure InternalStuff;
  end;
```

MCPConnect Offers



Attribute-Driven Development for Delphi
Equivalent registration code for C++Builder

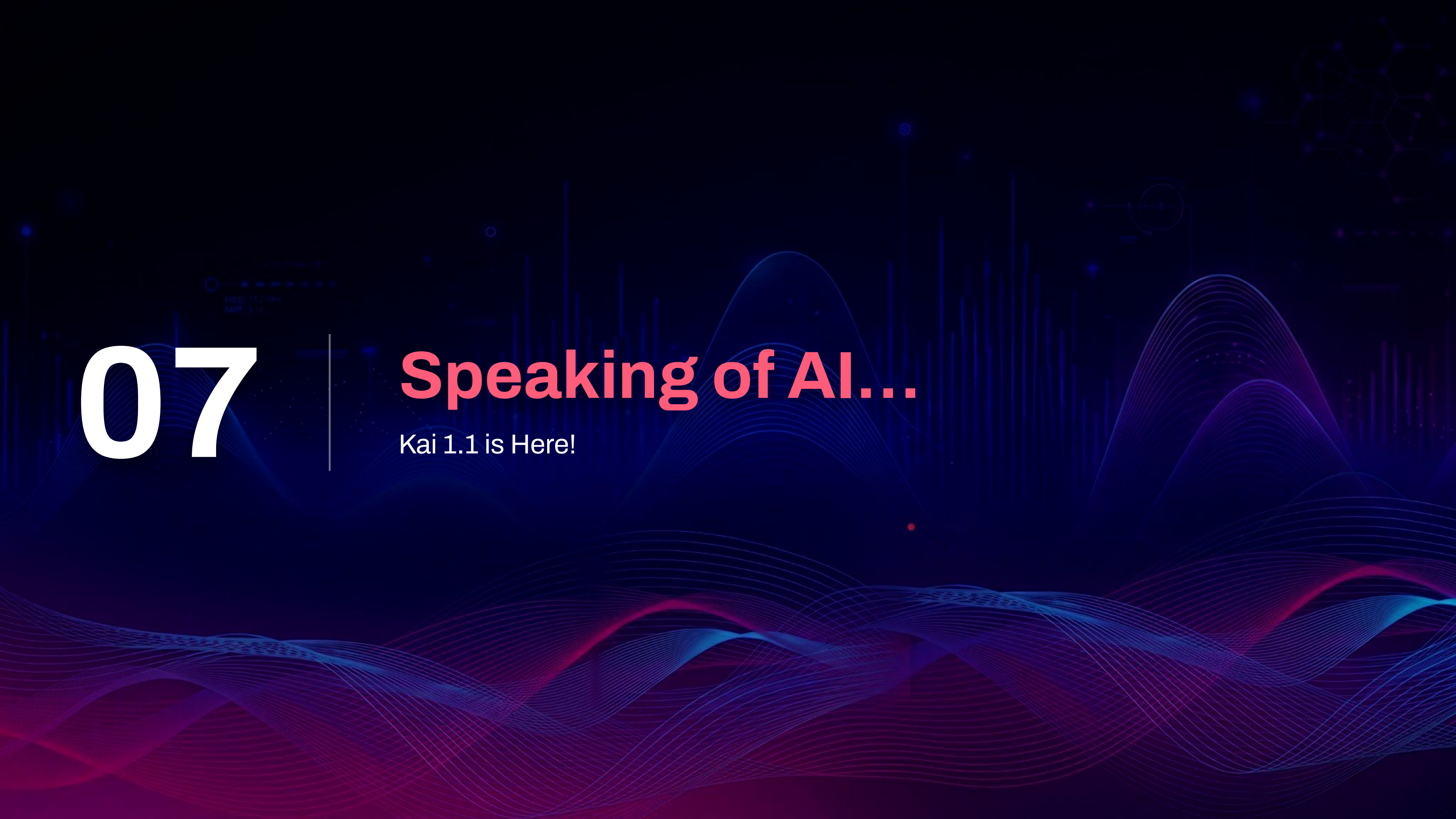


Multi-Transport & Session Management



Attribute-Driven Development for Delphi

Also available: a separate, ready-to-use “MCP server for FireDAC.”



07

Speaking of AI...

Kai 1.1 is Here!

Kai 1.1: ACP Registry



What is the ACP Registry?

- Public list of CLI AI engines supporting the ACP protocol
- Maintained by JetBrains and Zed
- 40 AI engines included
<https://agentclientprotocol.com/get-started/registry>

Antigravity

Grok

Mistral

Cursor

Devin

Kilo Code

Junie

And many more

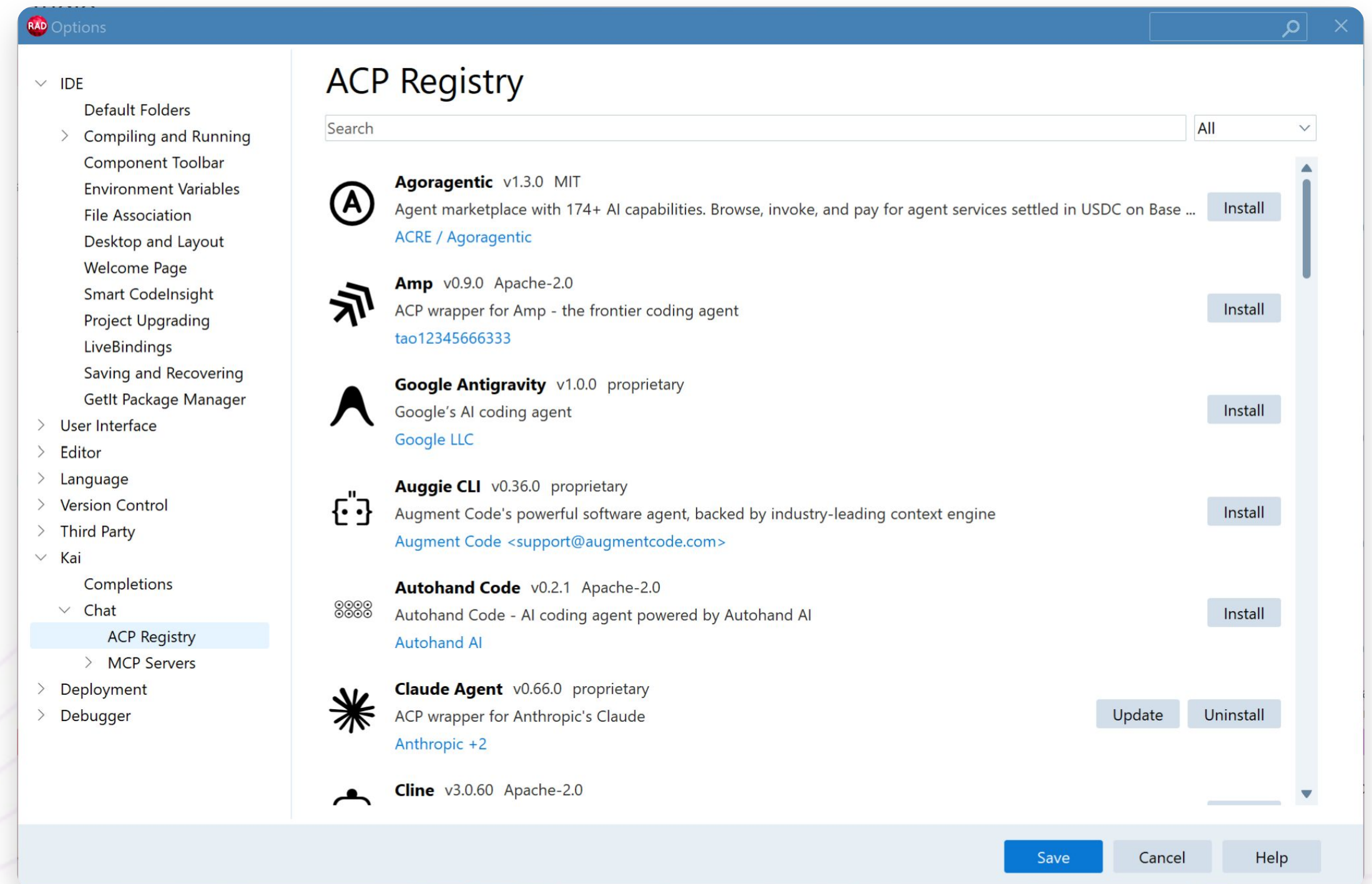
Kai 1.1 and ACP

- Kai was already using ACP
- Kai 1.1 offers full support for ACP Registry: install, uninstall, update
- No longer shipping specific CLI AI engines, besides:
 - Native LM Studio ACP agent for Ollama and LMStudio (for chat and completions)
 - Github CoPilot completion engine
- Authentication and sessions management are now fully ACP-based



Kai 1.1: ACP Registry

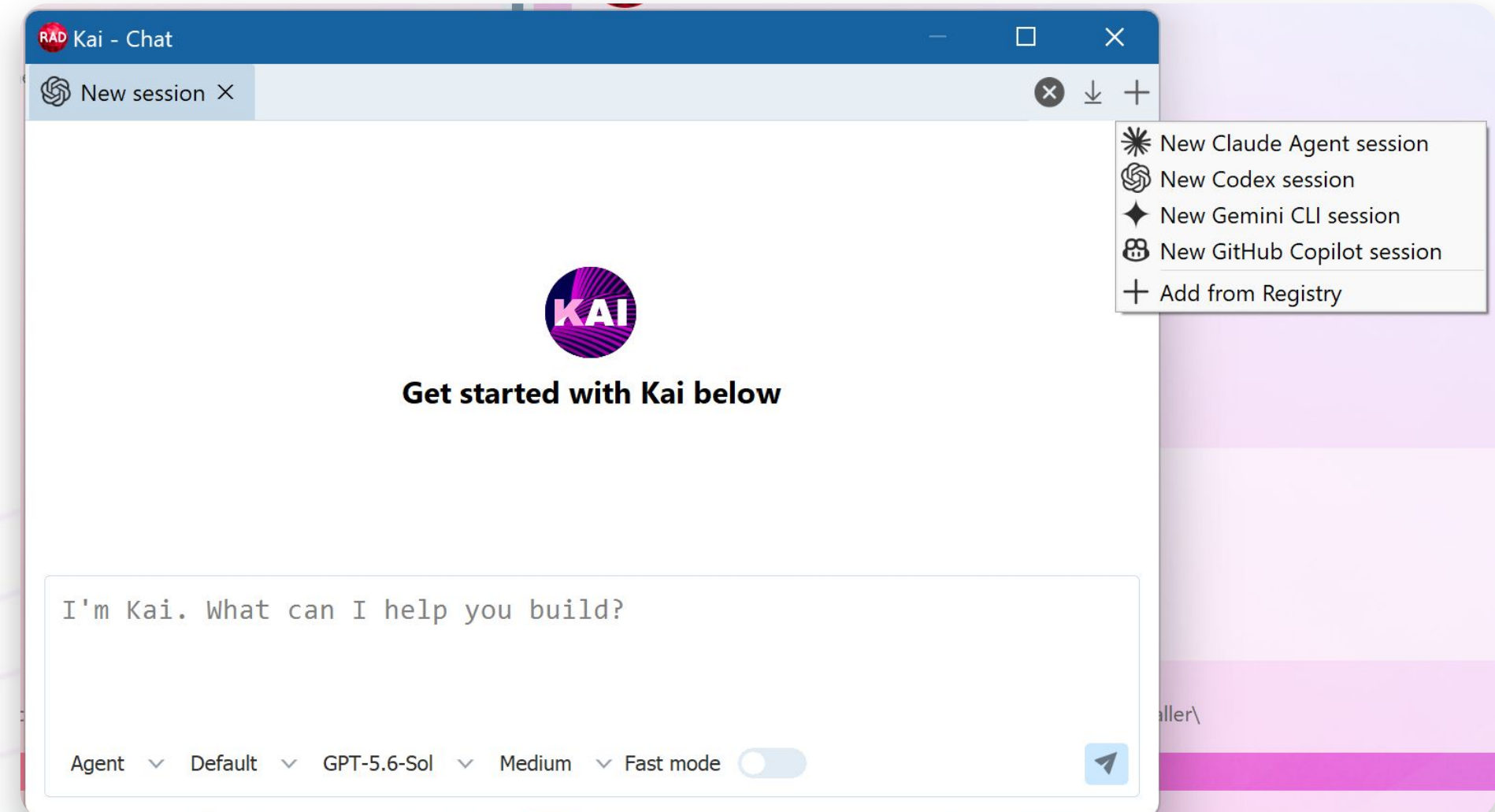
ACP registry can be managed in the Tools → Options dialog



Kai 1.1 UX Review



- Monochrome icons and glyphs
- Better spacing
- A new chat toolbar
- Cleaner configuration pages
More in line with Tools Options UX
- A more consistent editing chat area
- Improved boolean-option controls



Also in Kai 1.1



- Quality (dozens of customer reports addressed)
- Full control on Kai MCP endpoints
- You can disable operations you don't want agents to perform

Options

IDE

Default Folders

Compiling and Running

Component Toolbar

Environment Variables

File Association

Desktop and Layout

Welcome Page

Smart CodeInsight

Project Upgrading

LiveBindings

Saving and Recovering

GetIt Package Manager

User Interface

Editor

Language

Version Control

Third Party

Kai

Completions

Chat

ACP Registry

MCP Servers

Built-in

Custom

Deployment

Debugger

Built-in

IDE MCP Tools

Name	Description
☒ addBreakpoint	Adds a breakpoint at a given file and line num
☒ addFileToProject	Adds an arbitrary existing file to the specified
☒ applyEdit	Finds the exact text of oldContent in a file ope
☒ compileProjects	Compiles an array of given projects, waits for
☒ evaluateExpression	Evaluates a Delphi/C++ expression in the curr
☒ getActiveEditorFile	Returns the path of the currently active editor
☒ getCallStack	In case the current process has stopped, this i
☒ getEditorContent	Returns the current contents of a file as it is c
☒ getEditorLines	Returns a range of lines from a file as it is curr
☒ getModificationStat...	Returns whether a file is currently open in the
☒ getProjectInfo	Returns metadata about the specified project:
☒ getSelectedText	Returns the currently selected text from the u
☒ goToEditorPosition	Positions the cursor at a specific location in th
☒ listBreakpoints	Lists all breakpoints and returns the file path,
☒ listOpenFiles	Lists all files that are currently open in the RA
☒ openFile	Opens an existing file in the RAD Studio IDE fi
☒ openProject	Opens an existing RAD Studio project (.dproj,
☒ reloadFile	Reloads a currently open file from the hard di
☒ removeAllBreakpoin...	Removes all existing breakpoints. The respons
☒ removeBreakpoint	Removes a breakpoint in a given file and line.
☒ removeFileFromProj...	Removes a file from a RAD Studio project (wit
☒ renameFile	Renames a file in the active project of RAD St

Select All

Deselect All

Save

Cancel

Help

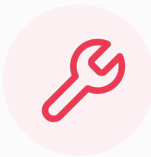


08

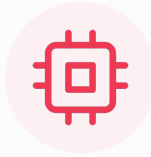
Quality Across the Product

Quality and fixes across all areas in RAD Studio

Quality Is a Release-Wide Theme



Long-standing IDE and editor problems addressed



Continued quality work for Windows Arm64EC and C++Builder Win64x



Significant debugger stability and performance improvements



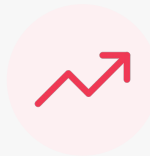
Fixes across VCL, FireMonkey, FireDAC, networking, deployment, and RAD Server

Customer-reported issues remain a major part of the release



103

customer support tickets addressed
80 were fixed



442

customer tickets resolved
339 fixed, 18 feature requests implemented

THANK YOU

RAD Studio 13.2 Florence: Available Now

